University of Toronto
Department of Computer Science

CSC263

Data Structures and Analysis

Lecture Notes

Lectures 1–6

Fall 2026

Instructor: Sam Toueg

Cheryl Shi

Contents

Lecture 1: Worst Case Time Complexity 4

1 Steps and Input Size 4

2 Worst Case Time Complexity 4

3 Proving Bounds on $T(n)$ 4

4 Two Issues $\Rightarrow$ Asymptotic Notation 5

5 Big O (Upper Bound) 5

6 Big Ω (Lower Bound) 5

7 Big Θ (Tight Bound) 6

8 Worked Example: Bubble Sort is $\Theta(n^2)$ 6

Lecture 2: Priority Queues and Max-Heaps 8

1 ADT vs. Data Structure 8

2 The Priority Queue ADT 8

3 Naive Implementations: Why We Need Something Better 8

4 Complete Binary Tree (CBT) 8
4.1 Height . . . 9

5 Max-Heap 9

6 Array Representation 10

7 The Three Operations 11
7.1 Max(A) . . . 11
7.2 Insert(A, x): percolate up . . . 11
7.3 Extract_Max(A): drip down . . . 12

8 Complexity Analysis 13

9 HeapSort 14

10 Past Exam: k-Set ADT 14

Quick Reference: Lecture 2 15

Lecture 3: Min Binomial Heaps 16

1 Mergeable Priority Queue ADT 16

2 Binomial Trees 16
2.1 Recursive Definition of B_k . . . 16

2.2 Children Sequence of B_k . . . 17
2.3 Properties of B_k . . . 18
2.4 Why "Binomial"? . . . 18

3 Binomial Forest F_n 18
3.1 Connection to Binary Representation . . . 18
3.2 What is $\alpha(n)$? . . . 19
3.3 Properties of F_n . . . 19

4 Min Binomial Heap: The Full Definition 19

5 Building a Binomial Heap: Worked Example 20

6 Memory Representation 21
6.1 Three Pointers Per Node . . . 21
6.2 Visual vs. Memory Representation . . . 21
6.3 HEAD Pointer and Root List . . . 21

7 Two Key Lemmas 22
7.1 Lemma 1: Merging Two B_k Trees . . . 22
7.2 Lemma 2: Deleting the Root of B_k . . . 23

8 Operations 23
8.1 Union(T, Q): Merging Two Binomial Heaps . . . 23
8.2 Insert(T, x): Adding a Single Element . . . 25
8.3 Min(T): Finding the Minimum . . . 25
8.4 Extract_Min(T): Remove the Minimum . . . 26

9 Complexity Summary 26

10 Practice Problems 27

11 Exam Relevance 28

Quick Reference: Lecture 3 28

Lecture 4: Binomial Heaps (Part 2) + BST 29

1 Remaining Binomial Heap Operations 29
1.1 Decrease_Key(T, x, k): Make a Key Smaller . . . 29
1.2 Delete(T, x): Remove a Specific Node . . . 30
1.3 Increase_Key(T, x, k): Make a Key Larger . . . 30
1.4 Complete Binomial Heap Operations Summary . . . 31

2 Cost of k Successive Inserts 31
2.1 Key Insight: Each Insert = Adding 1 in Binary . . . 32
2.2 Worked Example: 5 Successive Inserts into $|T| = 27$ . . . 32
2.3 Exact Formula via Edge Counting . . . 32

3 The Dictionary ADT 34
3.1 First Attempt: Linked List . . . 34

4 Binary Search Trees (BST) 34
4.1 The BST Property . . . 34

4.2 In-Order Traversal . . . 35
4.3 Left Rotation . . . 36
4.4 BST Operations . . . 37
4.5 BST Delete(T, x): Three Cases . . . 38
4.6 The Problem: Worst-Case Height . . . 41

5 Past Exam Questions **42**

Quick Reference: Lecture 4 **43**

Lecture 5: Balanced BSTs and AVL Trees **44**

1 Recall: the Problem with BSTs **44**

2 Height and Balance Factor **44**
2.1 Height of a Node . . . 44
2.2 Balance Factor . . . 44

3 AVL Trees **44**
3.1 Definition (Adelson-Velsky and Landis) . . . 45
3.2 Is This an AVL Tree? . . . 45
3.3 Properties . . . 45

4 AVL Operations **46**

5 Insert **46**
5.1 General Idea . . . 47
5.2 Worked Example 1: No Rebalancing . . . 47
5.3 Worked Example 2: Single Rotation . . . 48
5.4 Worked Example 3: Double Rotation . . . 48
5.5 The Four Rotation Cases . . . 49
5.6 Insert Algorithm Sketch . . . 50

Quick Reference: Lecture 5 **50**

Lecture 6: AVL Insert Correctness and Cost **51**

1 What This Lecture Adds **51**

2 Why the Insert Algorithm is Correct **51**
2.1 The Skipped Step: BF$(B) \neq 0$ . . . 51
2.2 Case 1(a): Single Left Rotation . . . 51
2.3 Case 1(b): Double Right-Left Rotation . . . 52

3 Cost, Node Layout, and Algorithm **53**
3.1 Rotation Cost and Total Insert Cost . . . 53
3.2 Contents of an AVL Node . . . 53
3.3 Final Insert Algorithm . . . 53

4 Exam-Style Practice **54**

Quick Reference: Lecture 6 **54**

Lecture 7: Augmenting Data Structures **55**

1 Augmenting a Data Structure **55**

2 Dynamic Order Statistics **55**
2.1 Example 55
2.2 Naive Attempt: Store Each Node's Rank 55

3 Size-Augmented AVL Tree **56**
3.1 The `size` Field 56
3.2 Select: the k-th Smallest 56
3.3 Rank of a Node 57
3.4 Maintaining `size` on Insert and Delete 58

4 Summary and Practice **58**

Quick Reference: Lecture 7 **59**

Lecture 1: Worst Case Time Complexity

1 Steps and Input Size

We count **basic operations**, each taking constant time:

Arithmetic	$+, -, *, /$
Comparisons	$>, <, =$
Data moves	load, store, copy

Input size n depends on the problem: number of array elements (sorting), number of vertices + edges (graphs), or number of bits (primality).

2 Worst Case Time Complexity

Worst case time complexity $T(n)$

Let A be an algorithm and $t(x) =$ steps A takes on input x.

$$T(n) = \max\{\, t(x) \mid x \text{ is an input of size } n \,\}$$

$T(n) =$ the **maximum** steps over *all* inputs of size n.

★ **Seen on a past exam** (Midterm, Fall 2014, Q1a)

```
FindLast(A, x):
    i ← A.length − 1
    while i ≥ 0 and A[i] ≠ x:
        i ← i − 1
    return i
```

Q. What is the worst case complexity of `FindLast`? (Count array accesses only.)

A. $T(n) = n$.
Worst case: $x \notin A$, so we scan every element exactly once $\Rightarrow$ n accesses. Cannot exceed n, so $T(n) = n$.

3 Proving Bounds on $T(n)$

The key: unpack what max means.

The max trick

$$\max(S) \le c \iff \forall e \in S : e \le c$$
$$\max(S) \ge c \iff \exists e \in S : e \ge c$$

Apply to $T(n) = \max\{t(x)\}$:

$T(n) \leq 7n^3 \iff$ for **every** input of size n, A takes **at most** $7n^3$ steps

$T(n) \geq 3n^2 \iff$ for **some** input of size n, A takes **at least** $3n^2$ steps

Example: proving $T(n) \geq 3n$ for FindLast

We need **some** input of size n with $t(x) \geq 3n$. Hmm, FindLast does at most n accesses, so $T(n) \geq 3n$ is **false**. But $T(n) \geq n$ is true: choose $x \notin A$, then $t(x) = n \geq n$. ✓

4 Two Issues ⇒ Asymptotic Notation

$T(n) \leq 7n^3$ vs. $T(n) \leq 100n^3$? The *shape* is n^3 either way. We want to ignore the constant.

The bound might fail for $n = 1, 2, 3$ but hold for all $n \geq 10$. We only need it for **sufficiently large** n.

Fix both ⇒ **asymptotic notation**: O, Ω, Θ.

5 Big O (Upper Bound)

Big O (formal)

$T(n)$ is $O(g(n)) \iff \exists\, c > 0,\ \exists\, n_0 > 0$ such that $\forall\, n \geq n_0$:

$$T(n) \leq c \cdot g(n)$$

For **every** input of size n (large enough), the algorithm takes **at most** $c \cdot g(n)$ steps.

Example: show $T(n) = 5n^2 + 3n$ is $O(n^2)$

Need c, n_0 such that $5n^2 + 3n \leq c \cdot n^2$ for all $n \geq n_0$.
For $n \geq 1$: $5n^2 + 3n \leq 5n^2 + 3n^2 = 8n^2$.
So $c = 8$, $n_0 = 1$. ✓

6 Big Ω (Lower Bound)

Big Ω (formal)

$T(n)$ is $\Omega(g(n)) \iff \exists\, c > 0,\ \exists\, n_0 > 0$ such that $\forall\, n \geq n_0$:

$$T(n) \geq c \cdot g(n)$$

For **some** input of size n (large enough), the algorithm takes **at least** $c \cdot g(n)$ steps.

O vs Ω in one line

O = **every** input, **at most**. Ω = **some** input, **at least**.

Example: show $T(n) = 5n^2 + 3n$ is $\Omega(n^2)$

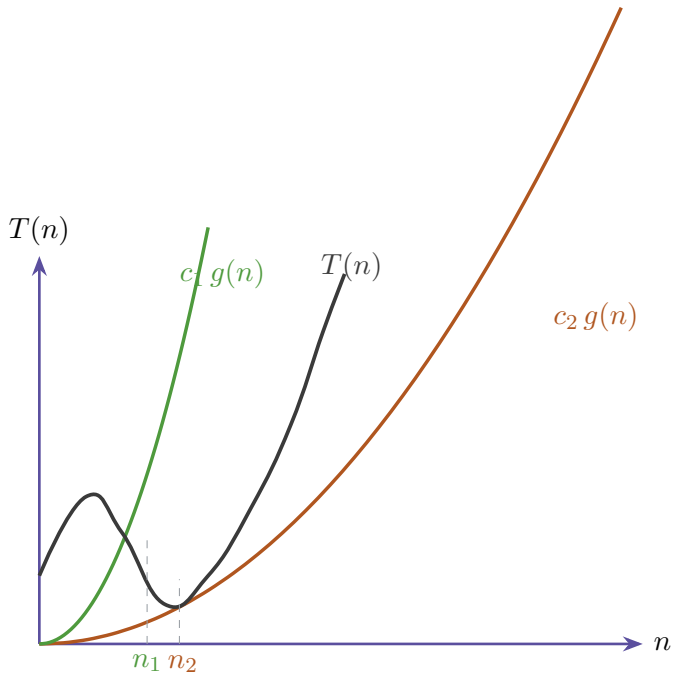
Need c, n_0 such that $5n^2 + 3n \geq c \cdot n^2$ for all $n \geq n_0$.
For $n \geq 1$: $5n^2 + 3n \geq 5n^2$.
So $c = 5$, $n_0 = 1$. ✓

7 Big Θ (Tight Bound)

Big Θ (formal)

$$T(n) \text{ is } \Theta(g(n)) \iff T(n) \text{ is } O(g(n)) \textbf{ and } T(n) \text{ is } \Omega(g(n))$$

Same $g(n)$ works as both upper bound and lower bound.



Past n_2, $T(n)$ is trapped between the $c_2 g(n)$ floor and the $c_1 g(n)$ ceiling. That is Θ.

8 Worked Example: Bubble Sort is $\Theta(n^2)$

BubbleSort(A)

```
for i = 1 to A.length − 1:
    for j = A.length downto i + 1:
        if A[j] < A[j−1]:  swap A[j] and A[j−1]
```

Bubble Sort: $T(n) = \Theta(n^2)$

$O(n^2)$: For **every** input, the nested loops do $\sum_{i=1}^{n-1}(n-i) = \frac{n(n-1)}{2}$ comparisons. So $T(n) \leq c_1 n^2$.
$\Omega(n^2)$: For **some** input (reverse-sorted: $[n, n-1, \ldots, 1]$), every comparison triggers a swap, doing exactly $\frac{n(n-1)}{2}$ comparisons. So $T(n) \geq c_2 n^2$.
Same shape $\Rightarrow \Theta(n^2)$.

Practice: is $3n^2 + 5n \log n$ in $\Theta(n^2)$?

$O(n^2)$: $3n^2 + 5n \log n \leq 3n^2 + 5n^2 = 8n^2$ for $n \geq 1$. ✓
$\Omega(n^2)$: $3n^2 + 5n \log n \geq 3n^2$ for $n \geq 1$. ✓
Yes, $3n^2 + 5n \log n = \Theta(n^2)$.

Quick Reference: Lecture 1

$T(n)$	$\max\{t(x) \mid x \text{ of size } n\}$	
$O(g(n))$	$T(n) \leq c\,g(n)$ eventually	**every** input, **at most**
$\Omega(g(n))$	$T(n) \geq c\,g(n)$ eventually	**some** input, **at least**
$\Theta(g(n))$	both O and Ω with same g (tight)	
Bubble sort	$\Theta(n^2)$	

Lecture 2: Priority Queues and Max-Heaps

1 ADT vs. Data Structure

Two different things

Abstract data type (ADT): specifies an object + operations. Says *what*, not *how*.
Data structure: one specific implementation of an ADT (memory layout + code for each operation).
Multiple data structures can implement the same ADT, with different costs.

2 The Priority Queue ADT

Priority Queue ADT

Object: a set S of elements, each with a comparable key (priority).
Operations:

- **Insert**(S, x): add element x into S.
- **Max**(S): return element with highest priority (no removal).
- **Extract_Max**(S): return **and remove** element with highest priority.

Application: OS job scheduling. Jobs arrive → **Insert**. Processor free → **Extract_Max** to run the highest-priority job.

3 Naive Implementations: Why We Need Something Better

Worst case	Insert	Extract_Max
Unordered linked list	$\Theta(1)$	$\Theta(n)$
Ordered linked list	$\Theta(n)$	$\Theta(1)$

Unordered: Insert at front is $\Theta(1)$, but Extract_Max scans the whole list $\Rightarrow \Theta(n)$.

Ordered: Max is at the end $\Rightarrow$ Extract_Max is $\Theta(1)$, but inserting in sorted position $\Rightarrow \Theta(n)$.

Goal

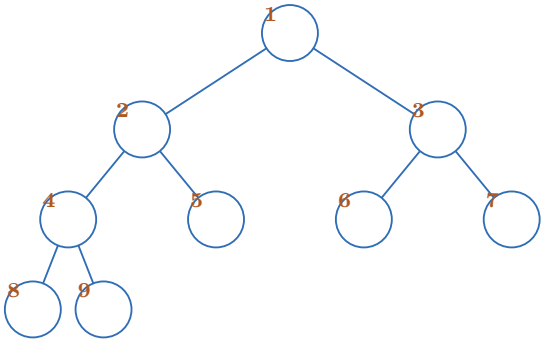
Both operations in $O(\log n)$. A **max-heap** achieves this.

4 Complete Binary Tree (CBT)

Complete Binary Tree

A binary tree where:

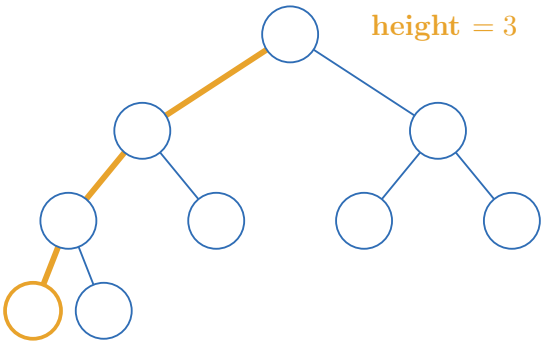
- every level except possibly the last is completely full,
- the last level is filled from the left with no gaps.



4.1 Height

Height of a tree

Number of edges on the longest root-to-leaf path.



Key fact

A CBT with n nodes has height $\lfloor \log_2 n \rfloor$.
Each full level doubles the node count $(1, 2, 4, 8, \ldots)$, so $\log_2 n$ levels suffice for n nodes.

5 Max-Heap

Max-Heap = CBT + max-heap property

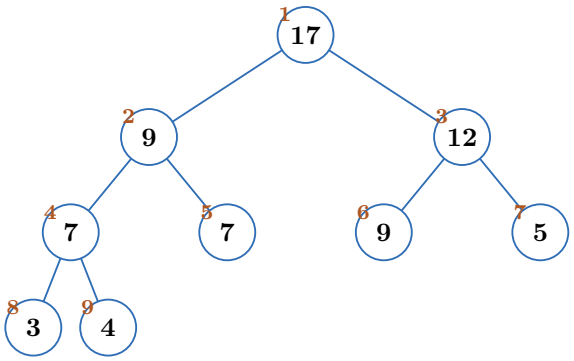
Elements stored in a CBT such that:

$$\text{priority(node)} \geq \text{priority(each child)}$$

This is a **local** rule (parent vs. own children only). It chains upward, so the **global max is always at the root**.

Example

$S = \{3, 4, 5, 7, 7, 9, 9, 12, 17\}$. One valid max-heap:

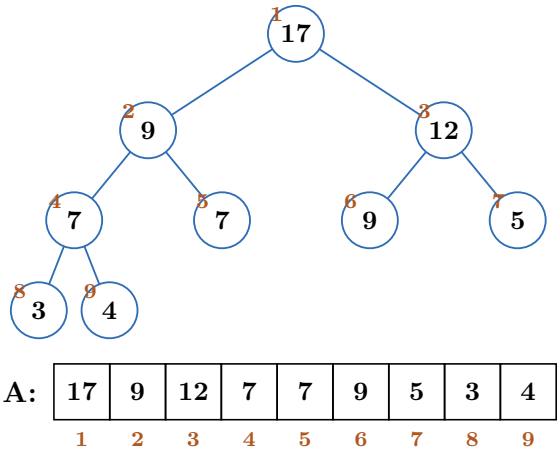


Check: $17 \geq 9, 12$. $9 \geq 7, 7$. $12 \geq 9, 5$. ✓

Swapping children of the root ($9 \leftrightarrow 12$) gives a different valid heap $\Rightarrow$ max-heaps are not unique. Only the root (global max) is guaranteed.

6 Array Representation

Store the tree **level by level, left to right** in array `A[1..n]`. No pointers needed.



Navigation formulas

For node at index i:

- **Left child:** $2i$ **Right child:** $2i + 1$ **Parent:** $\lfloor i/2 \rfloor$

`A.heapsize` tracks the number of elements currently in the heap.

Verify: node at $i = 2$ holds 9

Left child: $A[2 \cdot 2] = A[4] = 7$ ✓ Right child: $A[2 \cdot 2 + 1] = A[5] = 7$ ✓
Parent: $A[\lfloor 2/2 \rfloor] = A[1] = 17$ ✓
Each level is $\sim 2\times$ the width of the one above, so going down doubles the index, going up halves it.

7 The Three Operations

Invariant for every operation

1) Maintain the CBT shape. 2) Maintain the max-heap property.

7.1 Max(A)

Max(A)

```
return A[1]
```

The root always holds the max. $\Rightarrow \Theta(1)$.

7.2 Insert(A, x): percolate up

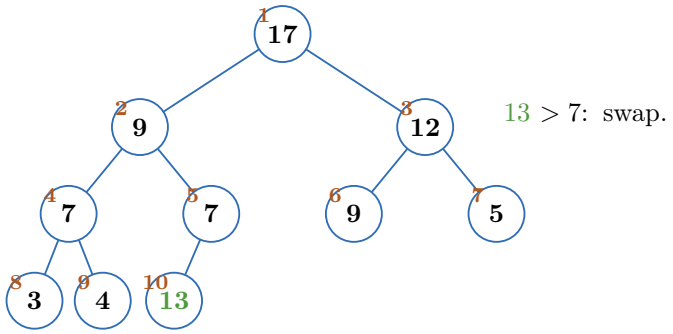
Insert(A, x)

```
1. A.heapsize++; A[A.heapsize] = x                    (place at next open leaf)
2. while x is not root and priority(x) > priority(parent(x)):
       swap x with parent(x)                          (percolate up)
```

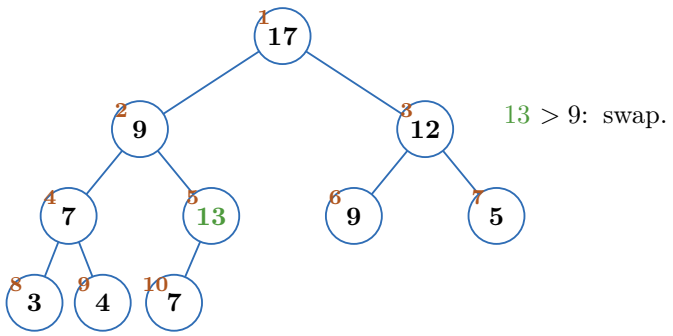
Worked example: Insert(A, 13)

Start: $A = [17, 9, 12, 7, 7, 9, 5, 3, 4]$. Insert 13 at index 10.

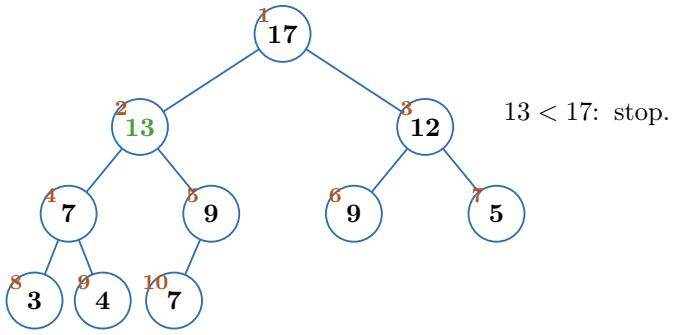
Step 1. Place 13 at $A[10]$. Property broken: $13 > 7$ (parent at $i = 5$).



Step 2a. Swap $13 \leftrightarrow 7$. Now 13 at $i = 5$. Compare: $13 > 9$ (parent at $i = 2$).



Step 2b. Swap $13 \leftrightarrow 9$. Now 13 at $i = 2$. Compare: $13 < 17$ (root). Stop.



Result: $A = [17, 13, 12, 7, 9, 9, 5, 3, 4, 7]$, `heapsize`$= 10$.

7.3 Extract_Max(A): drip down

Extract_Max(A)

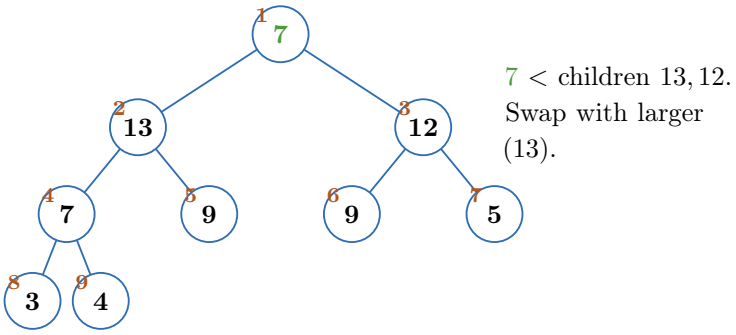
```
1. max = A[1]                                   (save the root)
2. A[1] = A[A.heapsize]; A.heapsize--           (move last leaf to root)
3. while x has a child with higher priority:
   swap x with its largest child                (drip down)
4. return max
```

Important: always swap with the *larger* child (otherwise we create a new violation). Node at index i is a leaf when $i > \lfloor$`heapsize`$/2 \rfloor$.

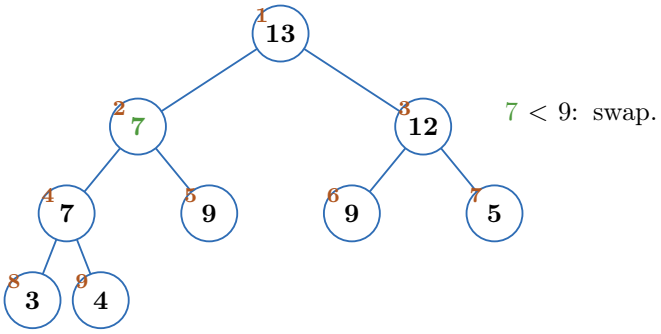
Worked example: Extract_Max(A)

Start: $A = [17, 13, 12, 7, 9, 9, 5, 3, 4, 7]$, `heapsize`$= 10$.

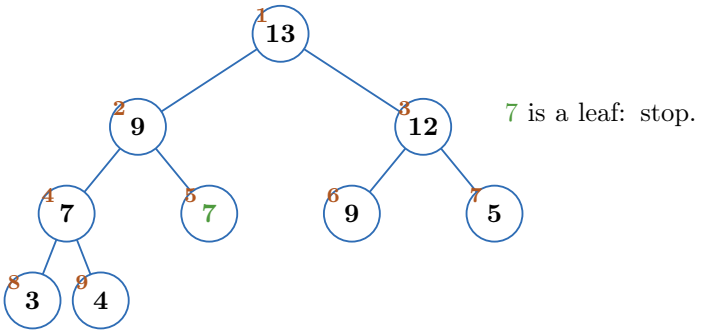
Steps 1 & 2. Save 17. Move $A[10] = 7$ to root. Shrink `heapsize` to 9.



Step 3a. Swap $7 \leftrightarrow 13$. Now 7 at $i = 2$. Children: $7, 9$. Larger is $9 > 7$.



Step 3b. Swap $7 \leftrightarrow 9$. Now 7 at $i = 5$. Children at $i = 10 >$ `heapsize`$= 9$, so 7 is a leaf. Stop.



Return 17. Heap: $[13, 9, 12, 7, 7, 9, 5, 3, 4]$.

8 Complexity Analysis

Each operation walks one root-to-leaf (or leaf-to-root) path. Path length $\leq$ height $= \lfloor \log_2 n \rfloor$. Constant work per level.

Heap operation costs

Max(A)	$\Theta(1)$	read root
Insert(A, x)	$\Theta(\log n)$	percolate up $\leq \lfloor \log_2 n \rfloor$ levels
Extract_Max(A)	$\Theta(\log n)$	drip down $\leq \lfloor \log_2 n \rfloor$ levels

Insert $= \Theta(\log n)$:

- $O(\log n)$: At most $\lfloor \log_2 n \rfloor$ swaps (one per level). **every** input, **at most** $c_1 \log n$.
- $\Omega(\log n)$: Insert a value larger than the current root $\Rightarrow$ percolates all the way to the top. **some** input, **at least** $c_2 \log n$.

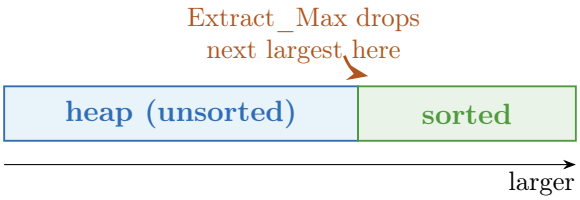
Same argument for Extract_Max (drip down with the smallest value $\Rightarrow$ sinks to a leaf).

9 HeapSort

HeapSort

1. Build a max-heap from A. ($\Theta(n)$ time)
2. Call **Extract_Max** n times. (each $\Theta(\log n)$)

Total: $n \times \Theta(\log n) = \Theta(n \log n)$



Each extraction moves the max to the front of the sorted region. The heap shrinks by one, the sorted region grows by one. After n extractions, the array is sorted in place.

10 Past Exam: k-Set ADT

★ **Seen on a past exam** (Midterm, Fall 2014, Q2)

The *rank* of an element = its position in sorted order (from 1). For fixed $k \in \mathbb{Z}^+$:

- **Object:** collection C of rationals (duplicates allowed).
- **Insert**(C, x): add x to C.
- k**-Select**(C): return element at rank k, or `nil` if $|C| < k$.

Q. Implement with data structures from class. k**-Select** must be $O(1)$.

Data structure: max-heap H_1 (stores the k smallest elements) + **min-heap** H_2 (stores the rest).
The k-th smallest element = the largest among the k smallest = root of H_1.

k**-Select**(C)**:**

```
if |H1| < k:  return nil
else:  return H1[1]                                   ⇒ O(1)
```

Insert(C, x)**:**

```
if |H1| < k or x < H1[1]:  insert x into H1
else:  insert x into H2
if |H1| > k:  insert Extract-Max(H1) into H2
```

Constant number of heap ops, each $O(\log n) \Rightarrow O(\log n)$.

Correctness: H_1 always holds exactly the k smallest. New small values enter H_1; overflow gets pushed to H_2. Large values go straight to H_2.

Quick Reference: Lecture 2

Max-heap property	priority(node) $\geq$ priority(children); max at root `A[1]`
Array navigation	left $= 2i$, right $= 2i+1$, parent $= \lfloor i/2 \rfloor$
Height of CBT	$\lfloor \log_2 n \rfloor$
Max	read `A[1]` $\to \Theta(1)$
Insert	place at end, percolate up $\to \Theta(\log n)$
Extract_Max	move last to root, drip down $\to \Theta(\log n)$
HeapSort	build heap + n extractions $\to \Theta(n \log n)$

Lecture 3: Min Binomial Heaps

1 Mergeable Priority Queue ADT

Last lecture: Priority Queue ADT (Insert, Max, Extract_Max). Now we flip to **min** (smallest key = highest priority) and add one more operation:

Mergeable Priority Queue ADT

Object: a set S of elements, each with a comparable key.
Operations:

- **Insert**(S, x): add element x to S.
- **Min**(S): return the element with the smallest key.
- **Extract_Min**(S): remove and return the element with the smallest key.
- **Union**(S_1, S_2): merge two priority queues into a single one.

Operation	Min Heap (array)	Min Binomial Heap
`Insert`	$O(\log n)$	$O(\log n)$
`Min`	$O(1)$	$O(\log n)$
`Extract_Min`	$O(\log n)$	$O(\log n)$
`Union`	$O(n)$	$O(\log n)$

A binary min heap stores elements in an array. To merge two heaps you basically dump them together and rebuild. That costs $O(n)$. Binomial heaps are specifically **designed** so Union runs in $O(\log n)$.

2 Binomial Trees

2.1 Recursive Definition of B_k

Binomial Tree B_k

Base case: B_0 = a single node.

Recursive case: B_k = take two copies of B_{k-1}. Make one root the **leftmost child** of the other root.

Equivalently: the root of B_k has children $B_{k-1}, B_{k-2}, \ldots, B_1, B_0$ (left to right).

Breaking Down the Recursive Definition

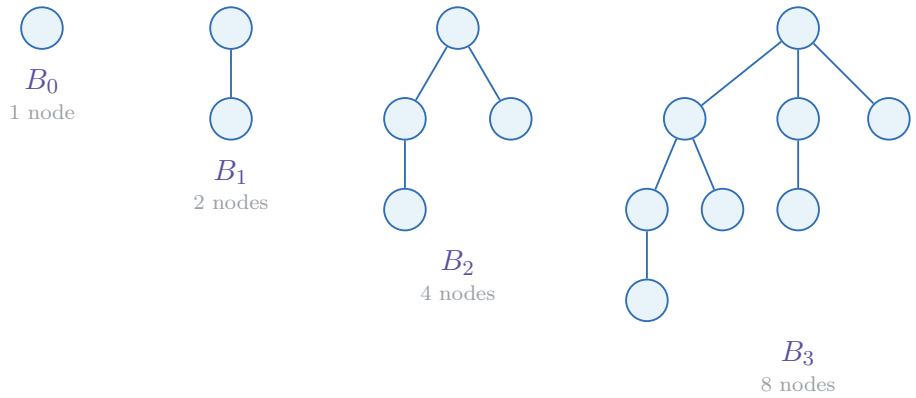
This is a common confusion point, so let's be explicit.
To build B_k:

1. Start with one copy of B_{k-1}. Call its root r.

2. Take a **second** copy of B_{k-1}.
3. Make the root of copy 2 the **new overall root**, and attach r's entire tree as a subtree under it.

So B_k = **two B_{k-1}'s glued together**, one becoming a child of the other.

Building Up From B_0



2.2 Children Sequence of B_k

Children of the Root of B_k

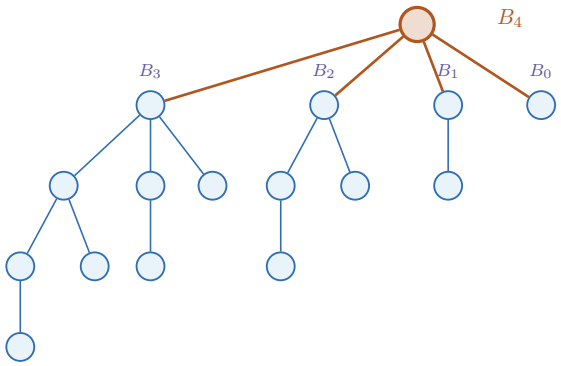
The root of B_k has exactly k children. Reading **left to right**:

$$\underbrace{B_{k-1}}_{\text{leftmost (biggest)}}, B_{k-2}, \ldots, B_1, \underbrace{B_0}_{\text{rightmost (single node)}}$$

The leftmost child is the largest subtree (B_{k-1}), and the rightmost child is the smallest (B_0). This ordering matters a lot for the memory representation later.

Visualizing B_4: Children of the Root

The root of B_4 has children B_3, B_2, B_1, B_0 (left to right, largest subtree to smallest). Total: $2^4 = 16$ nodes, height 4.



2.3 Properties of B_k

Key Properties of B_k

Height of B_k	$= k$
Number of nodes in B_k	$= 2^k$
Nodes at depth d in B_k	$= \binom{k}{d}$
Degree of root of B_k	$= k$

2.4 Why "Binomial"?

Why the Name "Binomial"

The number of nodes at depth d in B_k is $\binom{k}{d}$, the **binomial coefficient** (same "n choose k" from combinatorics).
Quick check for B_3 (8 nodes total):

$$\begin{aligned} \text{Depth 0:} \quad & \binom{3}{0} = 1 \quad \text{(the root)} \\ \text{Depth 1:} \quad & \binom{3}{1} = 3 \\ \text{Depth 2:} \quad & \binom{3}{2} = 3 \\ \text{Depth 3:} \quad & \binom{3}{3} = 1 \end{aligned}$$

Total: $1 + 3 + 3 + 1 = 8$ ✓

3 Binomial Forest F_n

Binomial Forest F_n

A **Binomial Forest** F_n is a sequence of binomial trees $\langle B_{k_1}, B_{k_2}, \ldots, B_{k_m} \rangle$ where:

- The k_i values are **strictly decreasing**: $k_1 > k_2 > \cdots > k_m$
- The total number of nodes across all trees $= n$

3.1 Connection to Binary Representation

Binary Representation ⇔ Which Trees Are Present

Write n in binary. B_i is present in F_n **if and only if** bit i (counting from the right, starting at 0) is **1**.
This is one of the most important ideas in this lecture. The entire structure of a binomial heap is dictated by the binary representation of n.

Example: F_7 and F_9

F_7: $7 = \langle 111 \rangle_2$
All three bits are 1, so $F_7 = \langle B_2, B_1, B_0 \rangle$.

Sizes: $4 + 2 + 1 = 7$ ✓

F_9: $9 = \langle 1001 \rangle_2$
Bits 0 and 3 are 1, so $F_9 = \langle B_3, B_0 \rangle$.
Sizes: $8 + 1 = 9$ ✓

3.2 What is $\alpha(n)$?

$\alpha(n)$ (popcount)

$\alpha(n)$ = the number of **1's** in the binary representation of n.
Other names: **popcount**, **Hamming weight**, **bit population count**.

Computing $\alpha(n)$

$$\begin{aligned} \alpha(7) &= \alpha(\langle 111 \rangle_2) &&= 3 \\ \alpha(9) &= \alpha(\langle 1001 \rangle_2) &&= 2 \\ \alpha(16) &= \alpha(\langle 10000 \rangle_2) &&= 1 \\ \alpha(15) &= \alpha(\langle 1111 \rangle_2) &&= 4 \end{aligned}$$

3.3 Properties of F_n

Properties of F_n

Number of trees in F_n	$= \alpha(n)$	(one per 1 bit)
Max number of trees	$\leq \lfloor \log_2 n \rfloor + 1$	
Number of edges in F_n	$= n - \alpha(n)$	

Verifying Edge Count for F_7

$F_7 = \langle B_2, B_1, B_0 \rangle$.
Edges in B_2: 3, edges in B_1: 1, edges in B_0: 0. Total = 4.
Formula: $n - \alpha(n) = 7 - 3 = 4$ ✓

4 Min Binomial Heap: The Full Definition

Min Binomial Heap

A **Min Binomial Heap** of n elements is a Binomial Forest F_n such that:

1. **One element per node:** each node stores exactly one element.
2. **Min heap ordering:** in each tree, every node's key $\leq$ all keys in its subtree. (Root of each tree = smallest key in that tree.)

A Min Binomial Heap is a **collection of trees** (a forest). Each individual tree is a binomial tree satisfying the min heap property. Do not confuse this with a single binary heap.

5 Building a Binomial Heap: Worked Example

Build a Min Binomial Heap from $S = \{10, 13, 1, 3, 8, 18, 7\}$

$n = 7$, and $7 = \langle 111 \rangle_2$, so the final forest $= \langle B_2, B_1, B_0 \rangle$.
Insert 10: $n = 1 = \langle 1 \rangle_2 \Rightarrow \langle B_0 \rangle$

$$B_0\text{: } \boxed{10}$$

Insert 13: $n = 2 = \langle 10 \rangle_2 \Rightarrow \langle B_1 \rangle$
Two B_0's $\rightarrow$ merge. Compare 10 vs 13: $10 \leq 13$, so 10 is root.

$$B_1\text{: } 10 \rightarrow 13$$

Insert 1: $n = 3 = \langle 11 \rangle_2 \Rightarrow \langle B_1, B_0 \rangle$
1 sits as the new B_0, B_1 stays.

$$B_1\text{: } 10 \rightarrow 13 \qquad B_0\text{: } \boxed{1}$$

Insert 3: $n = 4 = \langle 100 \rangle_2 \Rightarrow \langle B_2 \rangle$
3 forms B_0. Two B_0's (1 and 3): $1 \leq 3 \Rightarrow B_1$(root 1, child 3). Two B_1's (root 1 vs root 10): $1 \leq 10 \Rightarrow B_2$(root 1). 2 comparisons.

$$B_2\text{: root 1, children: } B_1(10 \rightarrow 13) \text{ and } B_0(3)$$

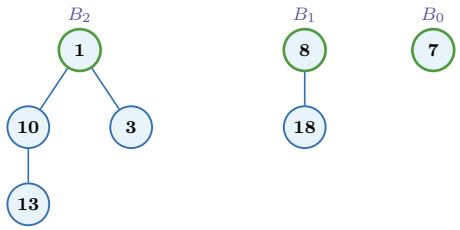
Insert 8: $n = 5 = \langle 101 \rangle_2 \Rightarrow \langle B_2, B_0 \rangle$

$$B_2(\text{root } 1) \qquad B_0\text{: } \boxed{8} \qquad 0 \text{ comparisons.}$$

Insert 18: $n = 6 = \langle 110 \rangle_2 \Rightarrow \langle B_2, B_1 \rangle$
Two B_0's (8 and 18): $8 \leq 18 \Rightarrow B_1$(root 8, child 18). 1 comparison.

$$B_2(\text{root } 1) \qquad B_1\text{: } 8 \rightarrow 18$$

Insert 7: $n = 7 = \langle 111 \rangle_2 \Rightarrow \langle B_2, B_1, B_0 \rangle$
7 is the new B_0. No merges needed. 0 comparisons.
Final heap: $\langle B_2(\text{root } 1),\ B_1(\text{root } 8),\ B_0(\text{root } 7) \rangle$



Total key comparisons: $0 + 1 + 0 + 2 + 0 + 1 + 0 = 4 = n - \alpha(n) = 7 - 3$

Important Facts

Each edge	corresponds to exactly **one key comparison**
Building from n elements	takes $O(n)$ key comparisons total

6 Memory Representation

6.1 Three Pointers Per Node

Pointer Structure

Each node stores **three pointers** plus two data fields:

Field	Points To	Color
PARENT	Parent node	Red
`key`	Element's key	(data)
`degree`	Number of children	(data)
CHILD	Leftmost child	Green
SIBLING	Right sibling	Blue

We need the PARENT pointer for Decrease Key (covered later).

6.2 Visual vs. Memory Representation

When we draw a binomial heap, the lines are **edges** (parent to child).
In memory, the actual **pointers** are:

- PARENT: child → its parent (upward)
- LEFT CHILD: parent → its **leftmost** child only
- RIGHT SIBLING: one child → the next sibling (across)

The visual edges are NOT pointers. Each visual edge is implemented through a combination of LEFT CHILD and RIGHT SIBLING pointers.

6.3 HEAD Pointer and Root List

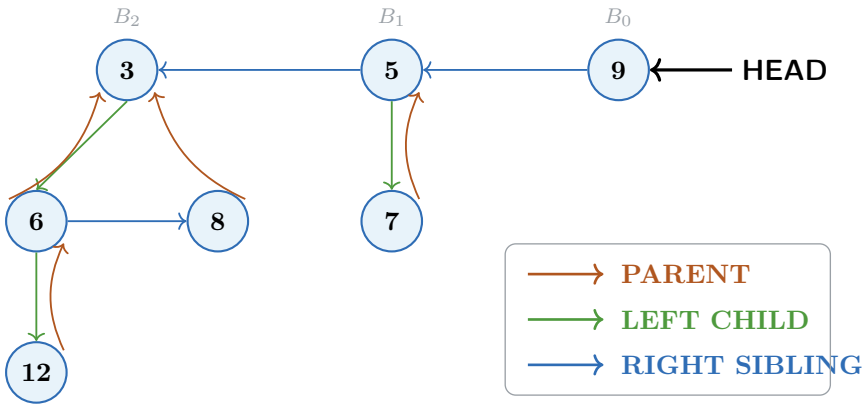
The roots of the B_k trees are connected via RIGHT SIBLING pointers, forming a **linked list of roots**. A **HEAD** pointer points to the root of the **smallest** tree (smallest k).

Memory Layout Example

Heap: $\langle B_3(\text{root } 8),\ B_2(\text{root } 1),\ B_0(\text{root } 7)\rangle$
Root linked list:

$$\texttt{HEAD} \to \boxed{7} \xrightarrow{\text{sib}} \boxed{1} \xrightarrow{\text{sib}} \boxed{8} \to \texttt{NIL}$$

HEAD points to 7 (B_0, smallest tree). Then 7's sibling pointer goes to 1 (B_2), then to 8 (B_3). Here is a full pointer diagram for a smaller heap $\langle B_2(\text{root } 3),\ B_1(\text{root } 5),\ B_0(\text{root } 9)\rangle$:



7 Two Key Lemmas

Before the operations, we need two building blocks. These lemmas are **the reason** all operations work efficiently.

7.1 Lemma 1: Merging Two B_k Trees

Lemma 1

We can merge two min heap ordered B_k trees into a single min heap ordered B_{k+1} tree using just one key comparison.

Why Lemma 1 Matters

This is the **carry operation**. Just like adding two 1 bits produces a carry in binary addition, merging two B_k's produces one B_{k+1}. This is the core mechanism behind Union, Insert, and Extract_Min.

Proof. Given two B_k trees with roots x and y:

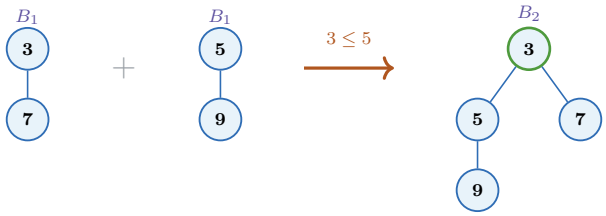
1. Compare x.key and y.key (one comparison).
2. Smaller root stays on top. Make the other tree the new leftmost subtree.

If $x \leq y$: x is root of B_{k+1}, with children $B_k(y)$, B_{k-1}, ..., B_0.

Min heap ordering preserved (smaller root on top). Structure is B_{k+1} by definition. □

Lemma 1 in Action: Merging Two B_1's into B_2

Two min heap ordered B_1 trees. One comparison decides the root.



$3 \leq 5$, so 3 stays as root. The other tree (root 5) becomes the **new leftmost child**. One comparison, one new edge.

7.2 Lemma 2: Deleting the Root of B_k

Lemma 2

Deleting the root of a min heap ordered B_k gives a valid min binomial heap of $2^k - 1$ elements.

Why Lemma 2 Matters

This is the key insight behind Extract_Min. When we extract the minimum (a root of some B_k tree), the remaining children form their own binomial heap. We then Union that new heap back with the rest.

Proof. Root of B_k has children $B_{k-1}, B_{k-2}, \ldots, B_1, B_0$.

Delete the root $\Rightarrow$ left with these k subtrees. Each is already min heap ordered (removing the root does not affect subtrees). The sequence $\langle B_{k-1}, \ldots, B_1, B_0 \rangle$ is a valid binomial forest (strictly decreasing k values), so it is a binomial heap of size $2^{k-1} + \cdots + 1 = 2^k - 1$. □

8 Operations

The Big Picture

Everything reduces to Union.

- Insert = Union with a tiny heap (single B_0)
- Extract_Min = remove a root (Lemma 2 gives a new heap) + Union
- Union itself = binary addition using Lemma 1 as the carry

Lemma 1 ($O(1)$ merge) and Lemma 2 (delete root $\Rightarrow$ valid heap) are the two pillars.

8.1 Union(T, Q): Merging Two Binomial Heaps

Union(T, Q)

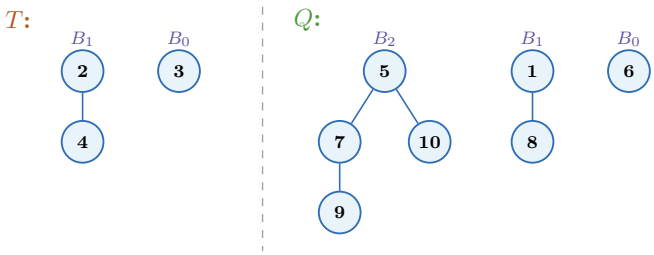
Input: two min binomial heaps T and Q.
Output: a single min binomial heap S containing all elements.
Algorithm: Binary addition!
Each column (bit position i) can have 0, 1, 2, or 3 copies of B_i (from T, from Q, plus possibly a carry from position $i - 1$). Process right to left:

- **0 trees**: nothing in S, no carry
- **1 tree**: put it in S, no carry
- **2 trees**: merge via Lemma 1 into B_{i+1}, carry it. Nothing in S at position i.
- **3 trees**: put one in S, merge the other two into a carry

Full Worked Example: Union(T, Q)

T has size $3 = \langle 11 \rangle_2$, so $T = \langle B_1, B_0 \rangle$.
Q has size $7 = \langle 111 \rangle_2$, so $Q = \langle B_2, B_1, B_0 \rangle$.
Result: $3 + 7 = 10 = \langle 1010 \rangle_2$, so $S = \langle B_3, B_1 \rangle$.
Concrete values:

- T: B_1(root 2, child 4). B_0(root 3).
- Q: B_2(root 5, children: 7→9 and 10). B_1(root 1, child 8). B_0(root 6).



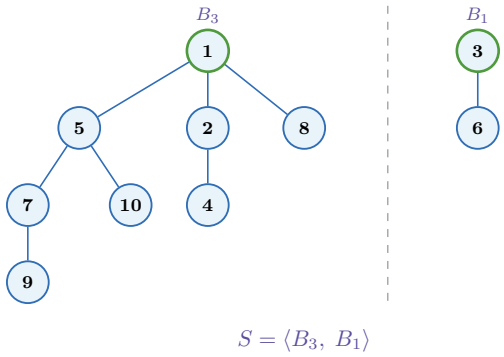
Position 0 (B_0 column):
T has $B_0(3)$, Q has $B_0(6)$. Two trees ⇒ merge.
Compare 3 vs 6: $3 \leq 6 \Rightarrow B_1$(root 3, child 6). This is the **carry**.
S at position 0: empty.

Position 1 (B_1 column):
T has B_1(root 2), Q has B_1(root 1), plus carry B_1(root 3). Three trees!
Keep one in S: the carry $B_1(3 \to 6)$.
Merge the other two: compare 1 vs 2: $1 \leq 2 \Rightarrow B_2$(root 1, with subtrees from both). New carry.
S at position 1: B_1(root 3, child 6).

Position 2 (B_2 column):
Q has B_2(root 5), plus carry B_2(root 1). Two trees ⇒ merge.
Compare 1 vs 5: $1 \leq 5 \Rightarrow B_3$(root 1). Carry to position 3.
S at position 2: empty.

Position 3 (B_3 column):
Just the carry B_3(root 1). One tree ⇒ done.
S at position 3: B_3(root 1).

Result: $S = \langle B_3(\text{root } 1),\ B_1(\text{root } 3) \rangle$ $\quad 10 = \langle 1010 \rangle_2$ ✓



New edges: 3 **Key comparisons:** 3

Union Complexity

If $|T| \leq n$ and $|Q| \leq n$, each heap has $\leq \lfloor \log_2 n \rfloor + 1$ trees.
$\Rightarrow$ Union takes $O(\log n)$ comparisons (one per column of the binary addition).

8.2 Insert(T, x): Adding a Single Element

Insert(T, x)

1. Create a new heap Q containing just x (a single B_0).
2. Return Union(T, Q).

Insert = Adding 1 in Binary

If the last bit is 0 (no B_0 exists): place the new B_0. Done in $O(1)$.
If the last bit is 1: merge into B_1 (carry). If B_1 already exists, merge again into B_2. This cascading carry can go up to $O(\log n)$ positions, but on average much less.

Insert Example

Heap $T = \langle B_2, B_0 \rangle$ (5 elements, binary 101). Insert element x.

1. x becomes B_0. Now two B_0's.
2. Merge them $\Rightarrow$ one B_1 (carry). No B_1 exists, so B_1 stays.
3. Result: $\langle B_2, B_1 \rangle$ (6 elements, binary 110). ✓

8.3 Min(T): Finding the Minimum

Min(T)

The minimum element is the root of one of the trees (by min heap property).
Walk through the root list (linked list via HEAD pointer). Compare each root's key. Return the smallest.
At most $\lfloor \log_2 n \rfloor + 1$ roots $\Rightarrow$ $O(\log n)$.

8.4 Extract_Min(T): Remove the Minimum

Extract_Min(T)

1. **Find the min:** walk the root list to find the tree B_k whose root has the smallest key. $O(\log n)$
2. **Remove that tree** from the heap T. $O(1)$
3. **Delete the root** of B_k. By **Lemma 2**, the children form a new binomial heap $Q = \langle B_{k-1}, \ldots, B_0 \rangle$. $O(\log n)$
4. **Union**(T, Q). $O(\log n)$

Total: $O(\log n)$.

Extract_Min Example

Heap: $\langle B_3(\text{root } 1),\ B_1(\text{root } 3) \rangle$ (10 elements)

1. Min $= 1$ (root of B_3).
2. Remove B_3 from heap. Remaining: $\langle B_1(\text{root } 3) \rangle$.
3. Delete root 1 from B_3. Children: B_2, B_1, B_0. New heap $Q = \langle B_2, B_1, B_0 \rangle$ (7 elements).
4. Union$(\langle B_1 \rangle, \langle B_2, B_1, B_0 \rangle)$:
   Position 0: one $B_0 \Rightarrow$ stays.
   Position 1: two B_1's $\Rightarrow$ merge into B_2 (carry).
   Position 2: two B_2's $\Rightarrow$ merge into B_3 (carry).
   Position 3: one $B_3 \Rightarrow$ stays.
   Result: $\langle B_3, B_0 \rangle$ (9 elements, $9 = \langle 1001 \rangle_2$) ✓

Return 1.

9 Complexity Summary

Binomial Heap Operations

Operation	Worst Case	Key Insight
`Union(`T`, `Q`)`	$O(\log n)$	binary addition
`Insert(`T`, `x`)`	$O(\log n)$	Union with single B_0
`Min(`T`)`	$O(\log n)$	scan root list
`Extract_Min(`T`)`	$O(\log n)$	Lemma 2 + Union

10 Practice Problems

Problem 1: Build a Binomial Heap

Build a min binomial heap by inserting: $5, 12, 3, 9, 2, 15, 6, 1$.
(a) How many trees does the final heap have?
(b) What is the root of each tree?
(c) How many total key comparisons?

$n = 8 = \langle 1000 \rangle_2 \Rightarrow F_8 = \langle B_3 \rangle$ (one tree).
Step by step:

- Insert 5: $\langle B_0(5) \rangle$. 0 comps
- Insert 12: merge B_0's: $5 \leq 12$. $\langle B_1(5 \to 12) \rangle$. 1 comp
- Insert 3: $\langle B_1(5), B_0(3) \rangle$. 0 comps
- Insert 9: merge B_0's: $3 \leq 9$. Two B_1's: $3 \leq 5$. $\langle B_2(\text{root } 3) \rangle$. 2 comps
- Insert 2: $\langle B_2(3), B_0(2) \rangle$. 0 comps
- Insert 15: merge B_0's: $2 \leq 15$. $\langle B_2(3), B_1(2 \to 15) \rangle$. 1 comp
- Insert 6: $\langle B_2(3), B_1(2), B_0(6) \rangle$. 0 comps
- Insert 1: merge B_0's: $1 \leq 6$. Two B_1's: $1 \leq 2$. Two B_2's: $1 \leq 3$. $\langle B_3(\text{root } 1) \rangle$. 3 comps

(a) 1 tree. (b) Root is 1. (c) $1 + 0 + 2 + 0 + 1 + 0 + 3 = 7 = n - \alpha(n) = 8 - 1$

Problem 2: Extract_Min

Given: $\langle B_2(\text{root } 3, \text{children: } 7 \to 9 \text{ and } 5),\ B_0(\text{root } 10) \rangle$. Perform Extract_Min.

1. Min $= 3$ (root of B_2). Extract it.
2. Remove B_2. Remaining: $\langle B_0(10) \rangle$.
3. Root 3 had children: B_1(root 7, child 9) and B_0(root 5). New heap $Q = \langle B_1(7 \to 9), B_0(5) \rangle$.
4. Union($\langle B_0(10) \rangle$, $\langle B_1(7), B_0(5) \rangle$):

   Position 0: two B_0's (10, 5). $5 \leq 10 \Rightarrow B_1$(root 5, child 10). Carry.

   Position 1: B_1(root 7) + carry B_1(root 5). $5 \leq 7 \Rightarrow B_2$(root 5). Carry.

   Position 2: just carry. Done.

   Result: $\langle B_2(\text{root } 5) \rangle$ (4 elements, $4 = \langle 100 \rangle_2$) ✓

Return 3.

Problem 3: Properties Quick Check

For B_5:

Nodes?	$2^5 = 32$
Height?	5
Degree of root?	5
Nodes at depth 2?	$\binom{5}{2} = 10$
Nodes at depth 5?	$\binom{5}{5} = 1$

For F_{13} ($13 = \langle 1101 \rangle_2$):

Which trees?	$\langle B_3, B_2, B_0 \rangle$
How many trees?	$\alpha(13) = 3$
How many edges?	$13 - 3 = 10$

11 Exam Relevance

Based on past CSC263 exams (2012 through 2015), no specific binomial heap questions appeared. However, heap questions show up frequently. The kSet ADT problem from Fall 2014 (see Lecture 2 notes) used max heaps and min heaps together.

- "Draw the binomial heap after inserting $x_1, x_2, \ldots$" $\Rightarrow$ binary addition method
- "Show the result of Extract_Min" $\Rightarrow$ Lemma 2 + Union
- "How many key comparisons?" $\Rightarrow$ count new edges = count merges
- "Why $O(\log n)$?" $\Rightarrow$ at most $\lfloor \log_2 n \rfloor + 1$ trees, binary addition has that many columns

Quick Reference: Lecture 3

B_k	two B_{k-1}'s merged; has 2^k nodes, height k, $\binom{k}{d}$ at depth d
F_n	forest of B_k's matching 1 bits of n; has $\alpha(n)$ trees, $n - \alpha(n)$ edges
Min Binomial Heap	F_n + one element per node + min heap ordering
Memory	3 pointers: PARENT, LEFT CHILD, RIGHT SIBLING; HEAD $\rightarrow$ root list
Lemma 1	merge two $B_k \rightarrow B_{k+1}$ with 1 comparison (the "carry")
Lemma 2	delete root of $B_k \rightarrow$ valid heap of $2^k - 1$ elements
Union	binary addition $\rightarrow O(\log n)$
Insert	Union with $B_0 \rightarrow O(\log n)$
Min	scan root list $\rightarrow O(\log n)$
Extract_Min	Lemma 2 + Union $\rightarrow O(\log n)$

Lecture 4: Binomial Heaps (Part 2) + Binary Search Trees

1 Remaining Binomial Heap Operations

Recall from Lecture 3: a Min Binomial Heap T is a forest of binomial trees $B_{k_1}, B_{k_2}, \ldots$, each satisfying the min-heap property (parent key $\leq$ children keys). Every node stores three pointers: PARENT, LEFT CHILD, RIGHT SIBLING.

We already have Union, Insert, Min, Extract_Min — all $O(\log n)$. Now we add three more operations that **modify a specific node** we already have a pointer to.

1.1 Decrease_Key(T, x, k): Make a Key Smaller

Goal: change the key of node x to a **smaller** value k (where $k < x$.key).

Why would we want this? Shortest-path algorithms (Dijkstra's) update distances as they find shorter paths. A priority queue needs to let you say "this element is now more urgent."

Decrease_Key(T, x, k)

Precondition: we have a **direct pointer** to node x. No searching needed.
Algorithm:

1. Set x.key $\leftarrow k$
2. Bubble up: while x is not a root **and** x.key $<$ x.parent.key:
   - swap x.key with x.parent.key
   - move x pointer up to the parent
3. Stop when min-heap property is restored (parent's key $\leq x$.key) or x is a root

Why "Bubble Up" Works

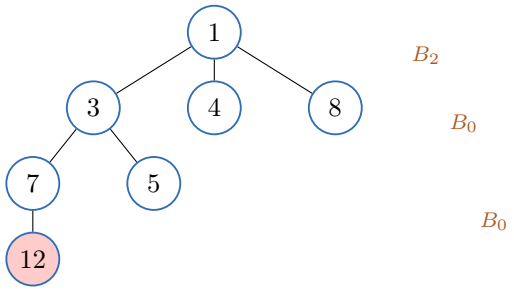
We decreased x's key, so the only possible violation is x.key $<$ x.parent.key (child smaller than parent). Swapping with parent fixes this violation but might create a new one further up. We keep going until we reach a parent whose key is $\leq$ ours, or we hit the root.
The PARENT pointer (from Lecture 3's memory representation) is what makes this efficient — we can walk straight up without searching.

Complexity: $O(\log n)$

At most we bubble up from the deepest node to the root. In a binomial tree B_k, the height is k. The largest tree in a heap of n elements is at most $B_{\lfloor \log_2 n \rfloor}$, so we do at most $\lfloor \log_2 n \rfloor$ swaps.

Worked Example: Decrease_Key

Suppose we have B_3 (8 nodes, height 3) with keys arranged as:



`Decrease_Key`$(T, x, 2)$ where x is the node with key 12:

1. Set x.key $= 2$
2. Compare with parent (key 7): $2 < 7 \Rightarrow$ swap. Now 2 is at depth 2, 7 is at depth 3.
3. Compare with parent (key 3): $2 < 3 \Rightarrow$ swap. Now 2 is at depth 1, 3 is at depth 2.
4. Compare with parent (key 1): $2 > 1 \Rightarrow$ STOP. Min-heap property restored.

Total swaps: 2 (out of max 3 = height of B_3). The tree is still a valid B_3 with min-heap ordering.

1.2 Delete(T, x): Remove a Specific Node

Goal: remove node x from T. Again, we have a direct pointer to x.

The trick: we already know how to remove a *root* (Extract_Min). So force x to become a root first!

Delete(T, x)

1. `Decrease_Key`$(T, x, -\infty)$: set x's key to $-\infty$. Since $-\infty$ is smaller than every other key, x bubbles **all the way up** to become the root of its binomial tree.
2. `Extract_Min`(T): the minimum element is now x (with key $-\infty$), so Extract_Min removes it. Internally this uses Lemma 2 (children of x's tree form a new heap) + Union.

Complexity: $O(\log n)$ (Decrease_Key $O(\log n)$ + Extract_Min $O(\log n)$)

1.3 Increase_Key(T, x, k): Make a Key Larger

Goal: change the key of node x to a **larger** value k (where $k >$ x.key).

When we *decrease* a key, the only violation is with the parent (one node above), and we bubble up along a single path.
When we *increase* a key, the violation is with the **children**. In a binomial tree, a node in B_k can have up to k children. To bubble down:

1. We need to find the **minimum-key child** among all children (to maintain min-heap property after swap).
2. A node in B_k can have up to k children $\Rightarrow$ scanning children costs $O(k)$ per level.
3. There are up to k levels to descend.

4. Total: $O(k^2) = O(\log^2 n)$. That is **worse** than $O(\log n)$!

Better approach — Delete + re-Insert:

Increase_Key(T, x, k)

1. `Delete`(T, x): remove x entirely. $O(\log n)$
2. Create a new node x' with key k.
3. `Insert`(T, x'): add x' back via Union. $O(\log n)$

Complexity: $O(\log n)$ — beats the naive bubble-down approach of $O(\log^2 n)$.

Worked Example: Increase_Key

Suppose T contains node x with key 3. We want `Increase_Key`$(T, x, 15)$.
Step 1: Delete(T, x)

- `Decrease_Key`$(T, x, -\infty)$: bubble x (now key $-\infty$) to root of its binomial tree
- `Extract_Min`(T): removes x. Children form a new heap (Lemma 2), Union it back.

Step 2: Create node x' with key 15.
Step 3: `Insert`(T, x') = `Union`$(T, \{x'\})$. The new node 15 enters as a B_0.
Total: $O(\log n) + O(\log n) = O(\log n)$.

1.4 Complete Binomial Heap Operations Summary

All Binomial Heap Operations — Final Table

Operation	Method	Cost
Union(T, Q)	binary addition (Lemma 1 = carry)	$O(\log n)$
Insert(T, x)	Union$(T, \{x\})$	$O(\log n)$
Min(T)	scan root list	$O(\log n)$
Extract_Min(T)	remove min root, Lemma 2 + Union	$O(\log n)$
Decrease_Key(T, x, k)	set key, bubble up via PARENT	$O(\log n)$
Delete(T, x)	Decrease_Key$(-\infty)$ + Extract_Min	$O(\log n)$
Increase_Key(T, x, k)	Delete + Insert (not bubble-down!)	$O(\log n)$

2 Cost of k Successive Inserts

We now ask a more refined question: if we insert k elements into a heap T one by one, how many **total** key comparisons does it take?

Setup: Start with binomial heap T where $|T| = n$. Insert k elements one by one.

Naive analysis: Each insert costs at most $O(\log(n + k))$. So k inserts $\leq O(k \log(n + k))$. But this is a loose overestimate — most inserts are cheap!

2.1 Key Insight: Each Insert = Adding 1 in Binary

Recall from Lecture 3: $\texttt{Insert}(T, x) = \texttt{Union}(T, \{x\})$, which works exactly like adding 1 in binary. The "carries" in binary addition correspond to tree merges (Lemma 1):

- No B_0 in T? (rightmost bit is 0) Just place x as B_0. 0 carries.
- B_0 exists? (rightmost bit is 1) Merge two B_0's $\to B_1$. That's 1 carry.
- B_1 already exists? Merge two B_1's $\to B_2$. Another carry.
- Keep carrying until we hit a 0-bit (an empty slot for that tree size).

Carries = Cost

Each carry = one key comparison (to decide which root goes on top when merging via Lemma 1). So the total cost of an insert = the number of carries it produces.

2.2 Worked Example: 5 Successive Inserts into $|T| = 27$

5 Successive Inserts into $|T| = 27$

$27 = \langle 11011 \rangle_2$ means T currently has trees B_4, B_3, B_1, B_0.
Watch what happens to the binary representation as we add 1 repeatedly:

Insert #	Binary before	Binary after	Carries	What happens
1	**11011**	**11100**	2	bit 0: $1+1 \to$ carry; bit 1: $1+\text{carry} \to$ carry; bit
2	11100	11101	0	bit 0 was 0, just place B_0
3	11101	11110	1	bit 0: $1+1 \to$ carry; bit 1: $0+\text{carry} \to 1$
4	11110	11111	0	bit 0 was 0, just place B_0
5	11111	100000	5	all 1's $\to$ carry ripples through all 5 positions
		Total:	**8**	

Naive bound: $5 \times \lfloor \log_2 32 \rfloor = 5 \times 5 = 25$. Actual: only **8**!
Notice: Insert #5 is expensive (5 carries) but inserts #2 and #4 are free. The expensive ones "pay" for all the 0-bits they create, making future inserts cheap.

2.3 Exact Formula via Edge Counting

We can derive an **exact** formula for the total cost using a clever counting argument.

Edge Counting Argument

Fact (from Lecture 3): a binomial heap with m elements has exactly $m - \alpha(m)$ edges, where $\alpha(m)$ = number of 1-bits in binary representation of m.
Setup:

- Before k inserts: n elements $\Rightarrow n - \alpha(n)$ edges
- After k inserts: $n + k$ elements $\Rightarrow (n + k) - \alpha(n + k)$ edges

- **New edges created** $= \big[(n+k) - \alpha(n+k)\big] - \big[n - \alpha(n)\big] = k + \alpha(n) - \alpha(n+k)$

Where do new edges come from? Each of the k inserted nodes gets connected to the tree (contributes 1 edge just by being placed). Each **carry** (Lemma 1 merge) creates exactly 1 additional edge (the new link between two merged roots). So:

$$\text{new edges} = \underbrace{k}_{\text{placed nodes}} + \underbrace{\text{total carries}}_{\text{merges}}$$

Wait — that gives: total carries $= k + \alpha(n) - \alpha(n+k) - k = \alpha(n) - \alpha(n+k)$.
But the **total cost** includes both the "place the node" step and the carries:

$$\boxed{\text{Total cost} = k + \alpha(n) - \alpha(n+k)}$$

Verify with the example: $n = 27$, $k = 5$. $\alpha(27) = \alpha(\langle 11011\rangle_2) = 4$. $\alpha(32) = \alpha(\langle 100000\rangle_2) = 1$. Total cost $= 5 + 4 - 1 = 8$. ✓ (matches our count above!)

Tight Bound: Total cost $\leq 2k$ when $k > \log_2 n$

Since $\alpha(n) \leq \lfloor \log_2 n \rfloor + 1$ (at most $\lfloor \log_2 n \rfloor + 1$ bits) and $\alpha(n+k) \geq 1$ (at least one 1-bit):

$$\text{Total cost} = k + \alpha(n) - \alpha(n+k) \leq k + \lfloor \log_2 n \rfloor + 1 - 1 = k + \lfloor \log_2 n \rfloor$$

When $k > \log_2 n$: total cost $< k + k = 2k$.

Average cost per insert $= \dfrac{\text{total cost}}{k} < \dfrac{2k}{k} =$ **2 key comparisons**
Even though a single insert can cost up to $O(\log n)$, on average each insert costs less than 2!

Practice Problem: Cost of Successive Inserts

Q: Start with $|T| = 13 = \langle 1101\rangle_2$. Perform 3 successive inserts. What is the total cost?

Solution using binary addition:

Insert #	Before	After	Carries
1	1101	1110	1
2	1110	1111	0
3	1111	10000	4
		Total:	**5**

Verify with formula: cost $= k + \alpha(n) - \alpha(n+k) = 3 + \alpha(13) - \alpha(16) = 3 + 3 - 1 = 5$. ✓
Verify with edge counting: $13 - 3 = 10$ edges before. $16 - 1 = 15$ edges after. New edges $= 5$. ✓

— **New Topic** —

We now shift from priority queues to a different ADT. Priority queues are great when you always want the min (or max), but what if you want to **search for a specific key**? That requires a **Dictionary**.

3 The Dictionary ADT

Dictionary ADT

Object: A set S of elements, each identified by a key.
Operations:

- **Search**(S, k): find and return the element with key k (or report it doesn't exist)
- **Insert**(S, x): add element x to S
- **Delete**(S, x): remove element x from S (given a pointer to x)

Dictionaries are everywhere: symbol tables in compilers, database indices, contact lists, DNS lookups. Any time you need to look something up by a key.

3.1 First Attempt: Linked List

Insert	$\Theta(1)$	prepend to head
Search	$\Theta(n)$	must scan entire list (key could be anywhere)
Delete	$\Theta(n)$	need to search first, then remove

Search takes $\Theta(n)$ because a linked list has **no ordering** — there is no way to skip ahead. We need a structure that lets us **eliminate half the candidates** at each step.

Goal

A data structure where *all three* dictionary operations run in $O(\log n)$ time. The **Binary Search Tree** is our first attempt at this.

4 Binary Search Trees (BST)

4.1 The BST Property

The idea: store elements in a binary tree, arranged so that at each node, we know which direction to go. This is the **binary search** idea applied to a tree structure.

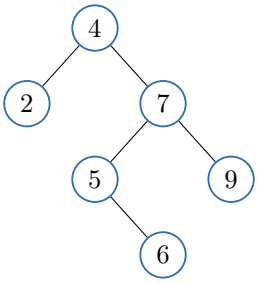
For **every** node x in the tree:

$$\text{keys in left subtree of } x \;\leq\; x.\text{key} \;\leq\; \text{keys in right subtree of } x$$

Note: Keys are **NOT necessarily unique**. The $\leq$ (not $<$) is critical — equal keys can go

either way.

Example: Valid BST with $S = \{2, 4, 5, 6, 7, 9\}$



Check the property at each node:

- **Node 4:** left subtree $\{2\}$ — all ≤ 4 ✓. Right subtree $\{5, 6, 7, 9\}$ — all ≥ 4 ✓.
- **Node 7:** left subtree $\{5, 6\}$ — all ≤ 7 ✓. Right subtree $\{9\}$ — $9 \geq 7$ ✓.
- **Node 5:** no left child. Right subtree $\{6\}$ — $6 \geq 5$ ✓.
- Leaves 2, 6, 9: no children, trivially satisfied.

The *shape* of the tree depends on the **insertion order**. The same set $S = \{2, 4, 5, 6, 7, 9\}$ can produce many different valid BSTs. This non-uniqueness is what leads to the worst-case problem we'll see later.

4.2 In-Order Traversal

A key consequence of the BST property: visiting nodes in **left-root-right** order gives all keys in ascending sorted order.

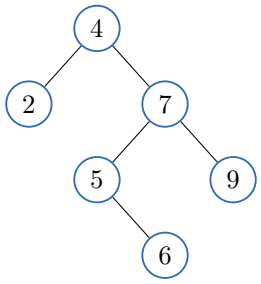
InOrderTraversal(x)

```
InOrderTraversal(x):
    if x != NIL:
        InOrderTraversal(x.left)          // recurse into left subtree
        visit(x)                          // process current node
        InOrderTraversal(x.right)         // recurse into right subtree
```

Why does this produce sorted order? The BST property guarantees: everything in the left subtree $\leq$ current node $\leq$ everything in the right subtree. Recursion applies this at every level.

Worked Example: In-Order Traversal

Using the BST above (root = 4):



Trace: go left from 4 → visit **2** → back up, visit **4** → go right → go left from 7 → visit **5** → go right from 5 → visit **6** → back up, visit **7** → go right from 7 → visit **9**
Result: $2, 4, 5, 6, 7, 9$ ✓ sorted!

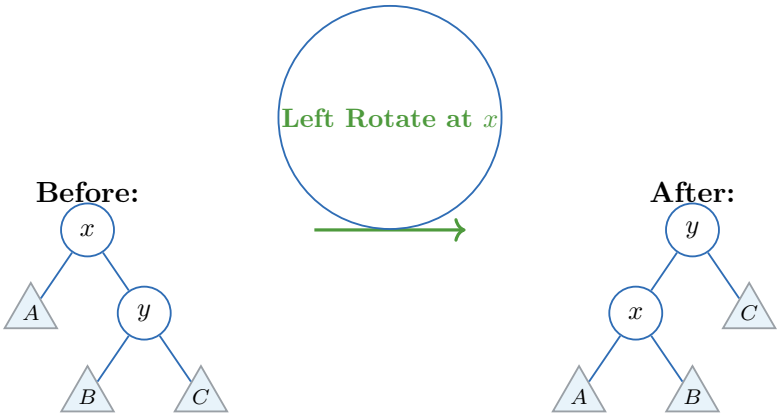
4.3 Left Rotation

A rotation restructures a BST without changing which keys it stores or violating the BST property. Rotations are the fundamental tool for **rebalancing** — we'll use them heavily when we get to AVL trees next lecture.

Left Rotation at node x

Let $y = x$.right (must exist). Left rotation at x:

1. y takes x's position (becomes new subtree root)
2. x becomes y's left child
3. y's old left subtree (B) becomes x's new right subtree

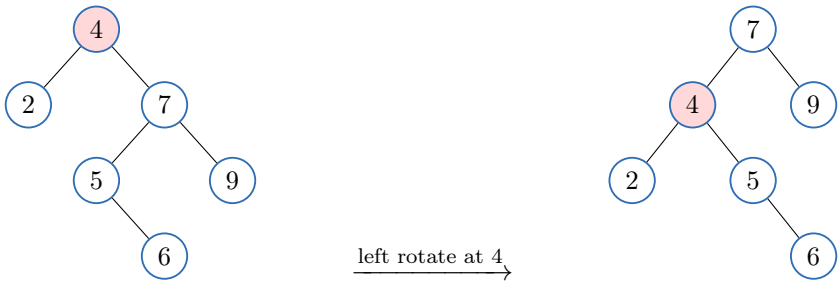


Why BST Property Is Preserved

Before the rotation, we know: keys in $A \leq x \leq$ keys in $B \leq y \leq$ keys in C.
After the rotation, B moves from y's left to x's right. But B's keys are between x and y, so B being x's right subtree ($\geq x$) and in y's left subtree ($\leq y$) is perfectly valid. Nothing else changes.
Cost: $O(1)$ — just rearranging 3 pointers.

Worked Example: Left Rotation at Node 4

$S = \{2, 4, 5, 6, 7, 9\}$, tree rooted at 4. Left rotate at node 4:
Before: root = 4, right child = 7, 7's left subtree = {5, 6}



7 becomes the new root. 4 becomes 7's left child. {5, 6} (was 7's left subtree) becomes 4's right subtree.
Verify: In-order traversal of the result: 2, 4, 5, 6, 7, 9. Same as before. BST property preserved. ✓

4.4 BST Operations

All BST operations follow one path from root toward a leaf, making comparisons at each node to decide left or right.

Search(T, k): Find a Key

Search(T, k)

```
x = T.root
while x != NIL and x.key != k:
    if k < x.key:
        x = x.left                          // k must be in left subtree
    else:
        x = x.right                         // k must be in right subtree
return x                                    // NIL if not found
```

Why this works: The BST property tells us that if $k < x$.key, then k cannot be in the right subtree (all keys there are $\geq x$.key $> k$). So we can safely ignore the entire right subtree and go left. Symmetric for $k > x$.key. This is the same idea as binary search on a sorted array.

Complexity: $O(h)$ where h = height of tree (we visit at most one node per level).

Worked Example: Search$(T, 5)$

Tree rooted at 4 → {2, 7 → {5 → {∅, 6}, 9}}:

1. At root 4: $5 > 4 \Rightarrow$ go right to 7
2. At 7: $5 < 7 \Rightarrow$ go left to 5
3. At 5: $5 = 5 \Rightarrow$ Found! Return this node.

3 comparisons, 3 nodes visited. Height of tree = 3, so this is within $O(h)$. ✓

Insert(T, z): Add a New Node

Insert(T, z)

```
Follow the search path for z.key (comparing at each node)
When you reach a NIL position, place z there as a new leaf
```

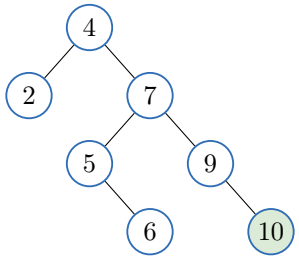
Why this works: The search path tells us exactly where z **should** be to maintain the BST property. The NIL position at the end is the only valid place for z's key. Since z becomes a leaf, it doesn't disturb any existing nodes.

Complexity: $O(h)$

Worked Example: Insert$(T, 10)$

Using the same tree. Search for where 10 belongs:

1. At root 4: $10 > 4 \Rightarrow$ go right
2. At 7: $10 > 7 \Rightarrow$ go right
3. At 9: $10 > 9 \Rightarrow$ go right — but right child of 9 is NIL!
4. Insert 10 as right child of 9.



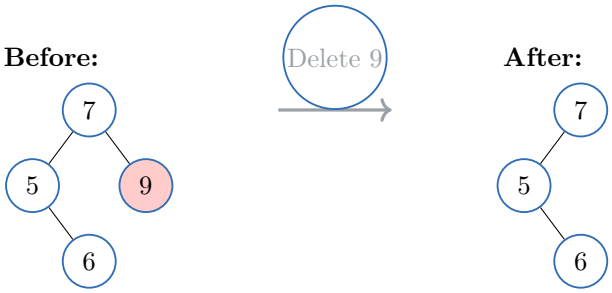
4.5 BST Delete(T, x): Three Cases

Deletion is the most complex BST operation. We have a pointer to node x and need to remove it while maintaining the BST property.

Removing a node from the middle of a tree can disconnect its children. We need to "repair" the tree so every node still has a parent and the BST property still holds.

Case 1: x is a leaf (no children)

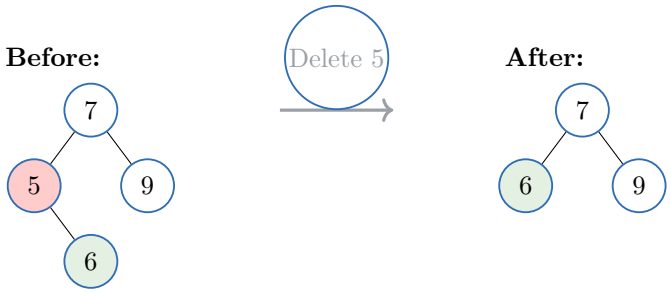
Simplest case. Just remove x — nothing to reconnect.



Case 2: x has exactly one child

Replace x with its only child (and the child's entire subtree). The child "moves up" to take x's position.

Why this is safe: if x had key k and its child's subtree contains keys that satisfied BST property relative to x, those keys also satisfy BST property relative to x's parent (they were already in the correct subtree of the parent).



Node 6 (the only child of 5) replaces 5. BST property: $6 \leq 7$ ✓.

Case 3: x has two children (hardest case)

We can't just remove x — both subtrees need a parent. The solution: replace x's key with its successor's key, then delete the successor (which is easier to remove).

The Successor

The successor of node x = the node with the **smallest key that is larger than** x.key. In a BST, the successor is in x's right subtree (since right subtree has all keys $\geq x$.key). To find it: go to x's **right child**, then follow **left child pointers all the way down** until you hit NIL. That leftmost node in the right subtree is the successor.

Why the Successor Has At Most One Child

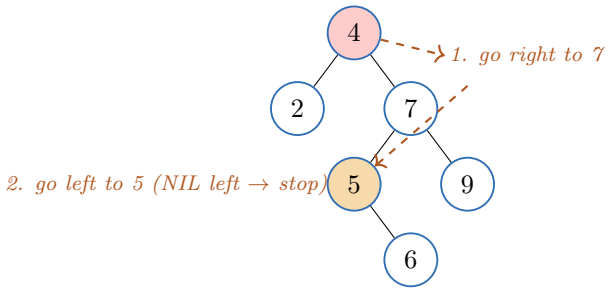
The successor is the **leftmost** node in some subtree. By definition, it has **no left child** (otherwise there would be a smaller key, contradicting "leftmost"). So the successor has at most one child (a right child). This means deleting the successor is either Case 1 or Case 2! This is the clever part: Case 3 **reduces to** Case 1 or 2.

1. **Find** the successor of x: go right once, then left all the way
2. **Copy** the successor's key into x's position (replacing x's key)
3. **Delete** the original successor node (Case 1 or 2)

Worked Example: Delete$(T, 4)$ — Two Children Case (Step by Step)

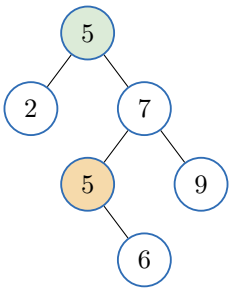
Tree: $4 \to \{2,\ 7 \to \{5 \to \{\emptyset, 6\},\ 9\}\}$
Sorted order of S: $2, \mathbf{4}, 5, 6, 7, 9$. The successor of 4 is 5.

Step 1: Find successor of 4

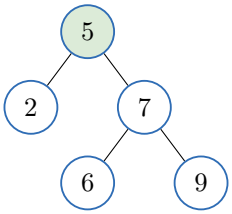


Go right to 7, then left to 5. Left child of 5 is NIL ⇒ successor = 5

Step 2: Copy successor's key into 4's position
Replace 4 with 5. Now there are temporarily two nodes with key 5.



Step 3: Delete original successor (the highlighted 5)
The successor node 5 has one child (6) ⇒ Case 2. Replace the successor 5 with its child 6:



Final result: $5 \to \{2,\ 7 \to \{6, 9\}\}$
Verify in-order: $2, 5, 6, 7, 9$ ✓ (4 removed, BST property intact)

Practice Problem: BST Delete

Q: Given BST: $8 \to \{3 \to \{1, 6 \to \{4, 7\}\},\ 10 \to \{\emptyset, 14 \to \{13, \emptyset\}\}\}$
Delete node 3 (two children).

Solution:

1. Successor of 3: go right to 6, then all-way left to 4. Successor = 4.
2. Copy 4 into 3's position.
3. Delete original 4: it is a leaf (Case 1), just remove it.
4. Result: $8 \to \{4 \to \{1, 6 \to \{\emptyset, 7\}\},\ 10 \to \{\emptyset, 14 \to \{13, \emptyset\}\}\}$

In-order: $1, 4, 6, 7, 8, 10, 13, 14$ ✓

BST Delete: Summary of All 3 Cases

Case	Condition	Action
1	x is a leaf (0 children)	Remove x
2	x has one child	Replace x with that child
3	x has two children	Copy successor's key into x, delete successor (Case 1 or 2)

All cases: $O(h)$. Finding successor walks down at most h nodes. Removing the successor is $O(1)$.

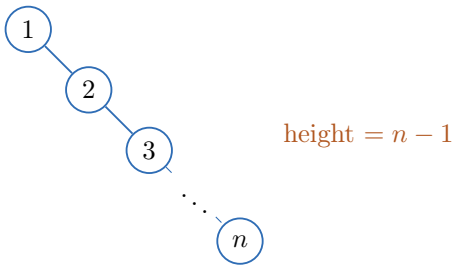
4.6 The Problem: Worst-Case Height

All BST operations run in $O(h)$ where h = height of the tree. So the real question is: **how tall can a BST get?**

Best case: A perfectly balanced BST with n nodes has height $\lfloor \log_2 n \rfloor$. All operations are $O(\log n)$.

Worst case: Consider inserting $1, 2, 3, \ldots, n$ **in sorted order**:

- Insert 1: root = 1
- Insert 2: $2 > 1$, goes right. Now: $1 \to 2$
- Insert 3: $3 > 1$, right. $3 > 2$, right. Now: $1 \to 2 \to 3$
- $\vdots$
- Insert n: goes right at every step. Chain: $1 \to 2 \to 3 \to \cdots \to n$



This is a degenerate chain — every node has exactly one child. It is a valid BST but it looks like a linked list!

Height $= n - 1$ edges. So all operations become $\Theta(n)$:

	Best-case BST	Worst-case BST
Search	$O(\log n)$	$\Theta(n)$
Insert	$O(\log n)$	$\Theta(n)$
Delete	$O(\log n)$	$\Theta(n)$

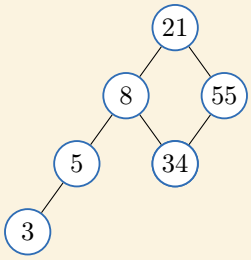
A plain BST gives us no guarantee on height. If the input happens to be sorted (or nearly sorted), we degrade to a linked list. We need a BST variant that **automatically rebalances** to keep the height small.

Next lecture: the AVL tree — a self-balancing BST that guarantees height $\leq O(\log n)$, making all dictionary operations $O(\log n)$ in the **worst case**. It uses **rotations** (like the left rotation we just learned) to rebalance after every insert and delete.

5 Past Exam Questions

★ **Seen on a past exam** (Fall 2014 Midterm, Q3b)

[4 marks] Draw the result when **55 is deleted** from the AVL tree below. Show your work; in particular, identify clearly any **rotation** performed.



Step 1: BST-Delete 55. Node 55 has one child (34) $\Rightarrow$ Case 2: replace 55 with 34.
Result: $21 \to \{8 \to \{5 \to \{3, \emptyset\}, 13\}, 34\}$
Step 2: Check AVL balance. Node 21 has left subtree height 2, right subtree height 0. Balance factor $= |2 - 0| = 2 > 1$. Unbalanced!
Step 3: Rotate. Right-rotate at 21 (with 8 as pivot).
Final result: $8 \to \{5 \to \{3, \emptyset\}, 21 \to \{13, 34\}\}$

Key concepts tested: BST Delete (Case 2) + AVL balance check + Rotation.
(AVL trees are covered in Lecture 5. This exam question is included to show what's coming and how BST deletion connects to AVL rebalancing.)

★ **Seen on a past exam** (Fall 2014 Midterm, Q2)

[10 marks] Consider the "k-Set" ADT:
Objects: Collection C of rational numbers (duplicates allowed).
Operations:

- `Insert`(C, x): add x to C
- `k-Select`(C): return element at rank k in C, or NIL if $|C| < k$

Implement using data structures from class. `k-Select` must run in $O(1)$ worst case.

Solution: Store k smallest elements in a max-heap H_1, rest in min-heap H_2.
Idea: the k-th smallest element = the largest among the k smallest = root of a max-heap containing exactly the k smallest.
`k-Select`(C): if $|H_1| < k$: return NIL; else return $H_1[1]$ (root of max-heap). $O(1)$.
`Insert`(C, x): if $x < H_1[1]$ or $|H_1| < k$: insert into H_1, else insert into H_2. If $|H_1| > k$: extract max from H_1, insert into H_2. Cost: $O(\log n)$.

Key concepts tested: Heap operations, ADT implementation, runtime analysis

Quick Reference: Lecture 4

Binomial Heaps (Part 2)

Decrease_Key	set key, bubble up via PARENT pointer $\to O(\log n)$
Delete	Decrease_Key$(-\infty)$ + Extract_Min $\to O(\log n)$
Increase_Key	Delete + Insert (not bubble-down: that's $O(\log^2 n)$) $\to O(\log n)$
k inserts total cost	$k + \alpha(n) - \alpha(n+k)$; $\leq 2k$ when $k > \log_2 n$
Average insert cost	< 2 key comparisons (when $k > \log_2 n$)

Dictionary ADT + BST

Dictionary ADT	Search + Insert + Delete on keyed elements
Linked list baseline	Insert $\Theta(1)$, Search/Delete $\Theta(n)$
BST property	left subtree keys $\leq$ node key $\leq$ right subtree keys
In-order traversal	left $\to$ visit $\to$ right = sorted order
Left rotation	x goes down, x.right goes up; preserves BST; $O(1)$
BST Search/Insert	follow comparisons down the tree; $O(h)$
BST Delete	3 cases: leaf / 1 child / 2 children (via successor); $O(h)$
Worst-case BST height	$n-1$ (degenerate chain) $\Rightarrow$ all ops $\Theta(n)$
Next: AVL trees	guarantee height $O(\log n)$ via rotations

Lecture 5: Balanced BSTs and AVL Trees

1 Recall: the Problem with BSTs

A plain BST implements the **Dictionary ADT** (Search, Insert, Delete), each in $\Theta(h)$ where h is the height. The catch: if we insert keys in sorted order $1, 2, 3, \ldots, n$, the tree becomes a degenerate chain with height $\Theta(n)$, so every operation is $\Theta(n)$.

Goal

Keep the tree **balanced** so its height stays $\Theta(\log n)$. Then Search, Insert, Delete are all $\Theta(\log n)$.

Balanced BST data structures for the Dictionary ADT: 2-3 trees, Red-Black trees, B-Trees, and AVL trees (this lecture).

2 Height and Balance Factor

2.1 Height of a Node

Height

height(v) = number of edges on the longest path from v down to a leaf.

- height(v) $= 0$ if v is a **leaf** (a single node).
- height(empty tree) $= -1$.

height(T) = height of the root node of T.

The -1 for the empty tree is a convention that makes the recurrence height(v) $= 1+\max($height(v.left), height(v.ri
work even when a child is missing.

2.2 Balance Factor

Balance Factor of a node v

$$\text{BF}(v) = \text{height(right subtree of } v) - \text{height(left subtree of } v) = h_R - h_L$$

The bigger $|\text{BF}(v)|$, the more imbalanced the node. We want every BF(v) as close to 0 as possible.

Reading the Balance Factor

$\text{BF}(v) = +1$ v is **right-heavy** $(h_R = h_L + 1)$
$\text{BF}(v) = \ \ 0$ v is **balanced** $(h_R = h_L)$
$\text{BF}(v) = -1$ v is **left-heavy** $(h_L = h_R + 1)$

3 AVL Trees

3.1 Definition (Adelson-Velsky and Landis)

An **AVL tree** T is a BST where **every** node $v \in T$ satisfies

$$-1 \leq \mathrm{BF}(v) \leq +1$$

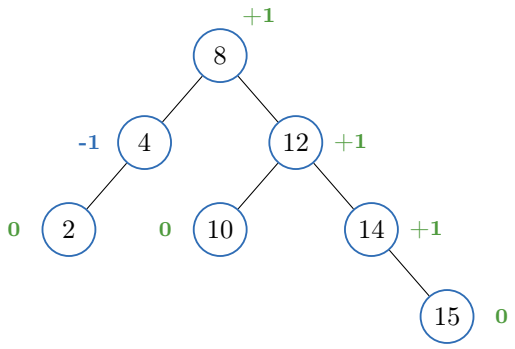
Every node is left-heavy, balanced, or right-heavy. **No node may reach** ± 2**.**

3.2 Is This an AVL Tree?

To check: compute $\mathrm{BF}(v)$ at *every* node. It is an AVL tree **iff** every $|\mathrm{BF}(v)| \leq 1$. A single node with $|\mathrm{BF}| \geq 2$ breaks it.

One tree, balance factor on every node

Compute $h_R - h_L$ at each node (heights shown are of the subtree rooted there).



Every $|\mathrm{BF}| \leq 1$, so this **is** an AVL tree. ✓

If any node reaches $\mathrm{BF} = +2$ or $\mathrm{BF} = -2$, the tree is not AVL. Example: a node whose left subtree has height 0 and right subtree has height 2 gives $\mathrm{BF} = 2 - 0 = +2$. Not AVL.

3.3 Properties

Why AVL trees are good

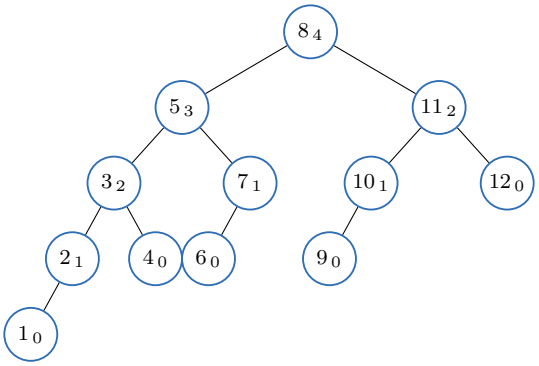
- An AVL tree of n nodes has height $\Theta(\log n)$ $\quad\left[\text{precisely, height} \leq 1.44 \log_2(n+2)\right]$.
- Insert and Delete can maintain the balance in $\Theta(\log n)$ time.
- Elegant, relatively clean and simple, works well in practice.

So Search, Insert, Delete are all $\Theta(\log n)$ worst case.

★ **Seen on a past exam** (Fall 2014 Midterm, Q3a)

[4 marks] Draw an AVL tree containing $\{1, 2, \ldots, 12\}$, but make it **as unbalanced as possible**. Next to each node, indicate the **height** of the subtree rooted at that node.

"As unbalanced as possible" = the **minimum** number of nodes are used to reach the height, so every node sits at its balance limit (BF = ± 1). One valid answer (height shown as a subscript):



Key idea: The subtree of height 4 (root 8) uses one child of height 3 and, by the AVL rule, the other child may be as short as height 2. That is the fewest nodes possible, which is why it looks so lopsided while still being AVL.

★ **Seen on a past exam** (Fall 2014 Midterm, Q3c)

[4 marks] Let $m(h)$ = the **minimum** number of nodes in any AVL tree of height h. Give a recurrence for $m(h)$ with base cases, and explain it briefly.

$$m(0) = 1, \qquad m(1) = 2, \qquad m(h) = 1 + m(h-1) + m(h-2) \text{ for } h \geq 2.$$

Why: height 0 needs 1 node; height 1 needs a root with one child (2 nodes). For height h, the root (1 node) needs one child of height $h-1$; by the AVL condition the *other* child can be as short as height $h-2$. Making both children minimal gives $m(h-1)+m(h-2)$ below the root.

Consequence: $m(h)$ grows like the Fibonacci numbers (exponentially in h), which forces $h = \Theta(\log n)$. This is the proof that AVL height is logarithmic.

4 AVL Operations

The three operations

Search(T, x)	identical to BST search $\to O(\log n)$ (height is $O(\log n)$)
Insert(T, x)	BST insert + rebalance $\to O(\log n)$
Delete(T, x)	BST delete + rebalance $\to O(\log n)$

Search is unchanged from a plain BST, so the only new work is keeping the tree balanced after Insert (and Delete). This lecture covers **Insert**.

5 Insert

5.1 General Idea

Insert(T, x): the plan

1. Insert x into T as in any BST. Now x is a **leaf**; set BF$(x) = 0$.
2. Walk **from x up to the root**. Only the balance factors of nodes *on this path* can change.
3. At each node v on the path, **adjust** BF(v). If some v becomes ± 2, **rebalance** at v with a rotation.

Why only the path changes: inserting a leaf only lengthens paths that pass through x's ancestors. No other node's subtree heights move.

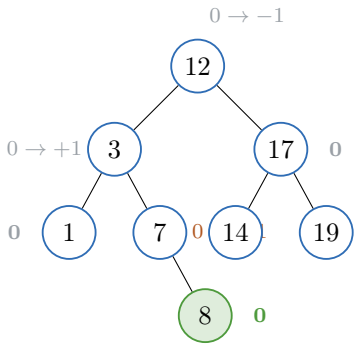
The stopping rule

Going up, at each ancestor v:

- If the subtree containing x got taller, BF(v) moves by ± 1.
- **If BF(v) becomes** 0, the subtree at v did *not* grow taller overall $\Rightarrow$ stop. Nothing above v can change.
- If BF(v) becomes ± 2, rotate to fix it (then the height is restored, so we also stop).

5.2 Worked Example 1: No Rebalancing

Start from this AVL tree of $\{1, 3, 7, 12, 14, 17, 19\}$ (all BF $= 0$), and **Insert 8**.



Trace: Insert 8

1. BST-insert 8: it lands as the **right child of 7**. Set BF$(8) = 0$.
2. Go up to 7: 8 is in 7's right subtree $\Rightarrow$ BF$(7) : 0 \to +1$. Still legal, and **the height of the subtree at 7 did not change** (it was 1 before, it is still 1). So we stop at 7.

Because the height of 7's subtree is unchanged, no ancestor above it is affected (3 and 12 keep their old balance factors, shown greyed). No rebalancing needed.

After inserting 8, node 3 would only change if 7's subtree grew. It did not, so we never even reach 3: we stop the moment a node keeps its height. This is the whole point of the stopping rule.

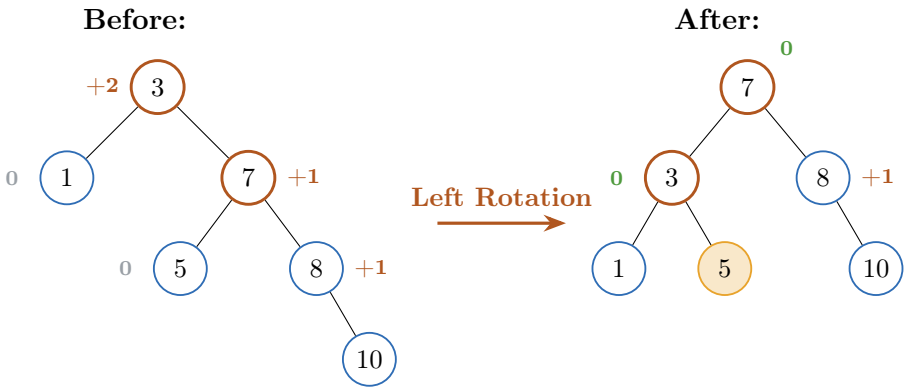
5.3 Worked Example 2: Single Rotation

Now the tree is the AVL of $\{1, 3, 5, 7, 8, 12, 14, 17, 19\}$. **Insert 10.**

10 BST-inserts as the **right child of 8**. Walking up: BF(8) : $0 \to +1$, then BF(7) : $0 \to +1$, then BF(3) : $+1 \to +2$. Node 3 is now unbalanced (right-heavy), and its right child 7 is also right-heavy (+1).

This is the "right-right" (RR) case ⇒ single Left Rotation

$v = 3$ has BF(v) $= +2$ **and** BF(v.right) $= +1$. Fix with one **Left Rotation** at v.



Reading the rotation

Left-rotating at 3: its right child 7 becomes the new subtree root, and 3 becomes 7's **left** child.

- **Where does 5 go?** 5 was 7's *left* subtree. In a left rotation, 7's old left subtree becomes 3's new **right** subtree. So 5 hangs under 3, not under 8, because $3 < 5 < 7$, so 5 belongs on 3's right. (BST property preserved.)
- After: BF(7) $= 0$, so 7's subtree is balanced again.

Bonus: nothing propagates upward

The subtree rooted at 3 had height 2 *before* inserting 10; after the rotation the subtree rooted at 7 has height 2 again. **Same height as before the insert**, so the balance factor of 12 (the parent) is unchanged and we stop. One rotation fixes everything.

5.4 Worked Example 3: Double Rotation

Tree is the AVL of $\{1, 3, 5, 7, 8, 12, 14, 17, 19\}$. **Insert 6.**

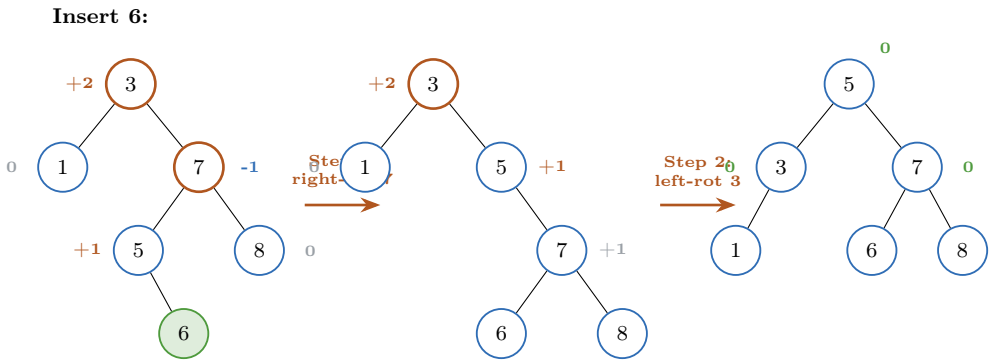
6 BST-inserts as the **right child of 5**. Walking up: BF(5) : $0 \to +1$, BF(7) : $0 \to -1$, BF(3) : $+1 \to +2$. Now $v = 3$ has BF $= +2$, but its right child 7 is left-heavy (-1): the "bend" zig-zags.

Because the heavy grandchild is on the *left* of the right child (a right-then-left zig-zag), one left rotation at 3 leaves the tree unbalanced. We need **two** rotations.

This is the "right-left" (RL) case ⇒ Double Right-Left Rotation

$v = 3$ has $\mathrm{BF}(v) = +2$ **and** $\mathrm{BF}(v.\mathrm{right}) = -1$.

1. **Right-rotate** the right subtree of v (rotate around 7).
2. **Left-rotate** at v (rotate around 3).



Result of the double rotation

The middle key 5 rises to the top of the subtree, with 3 on its left and 7 on its right. Every node is back to BF = 0, the tree is still a BST, and the subtree height matches what it was **before** inserting 6, so we stop: no rebalancing propagates to 12.

5.5 The Four Rotation Cases

At the lowest unbalanced node v (with $\mathrm{BF}(v) = \pm 2$), which fix to apply is decided by the balance factor of the child on the heavy side.

Rotation case table

$\mathrm{BF}(v)$	heavy child's BF	Shape	Fix
$+2$	$\mathrm{BF}(v.\mathrm{right}) = +1$	right-right (RR)	single Left rotation at v
$+2$	$\mathrm{BF}(v.\mathrm{right}) = -1$	right-left (RL)	Right rotation on v.right, then Left at v
-2	$\mathrm{BF}(v.\mathrm{left}) = -1$	left-left (LL)	single Right rotation at v
-2	$\mathrm{BF}(v.\mathrm{left}) = +1$	left-right (LR)	Left rotation on v.left, then Right at v

Rule of thumb

- Heavy side and grandchild's heavy side **agree** (both right, or both left) ⇒ single rotation.
- They **disagree** (a zig-zag) ⇒ double rotation (fix the child first, then v).

After the rotation the subtree's height returns to what it was before the insert, so **at most one rotation (single or double) is ever needed per insert**.

5.6 Insert Algorithm Sketch

Insert(T, x)

```
1.  Insert x into T as in any BST.
    x is now a leaf; set BF(x) = 0.
2.  Go up from x to the root.  For each node v on this path:
    Adjust BF(v):
       if x is in the RIGHT subtree of v:  increment BF(v)
       if x is in the LEFT subtree of v:  decrement BF(v)
       if BF(v) == 0:   (subtree height unchanged)  STOP
    Rebalance if necessary:
       if BF(v) == +2:
           if BF(v.right) == +1:  Left rotation; update BFs; STOP
           if BF(v.right) == -1:  Right-Left rotation; update BFs; STOP
       if BF(v) == -2:   (symmetric: Right or Left-Right rotation) STOP
```

Cost: the walk up is at most the height $= O(\log n)$, each step is $O(1)$, and at most one rotation ($O(1)$) happens. So **Insert is** $O(\log n)$.

Quick Reference: Lecture 5

Height	edges to deepest leaf; leaf $= 0$, empty $= -1$
BF(v)	$h_R - h_L$; $+1$ right-heavy, 0 balanced, -1 left-heavy
AVL tree	BST with $-1 \le \text{BF}(v) \le +1$ for *every* node
AVL height	$\Theta(\log n)$ $(\le 1.44 \log_2(n+2))$; $m(h) = 1 + m(h-1) + m(h-2)$
Search	same as BST $\to O(\log n)$
Insert	BST-insert leaf, walk up adjusting BF, rotate if $\pm 2 \to O(\log n)$
Stop rule	stop when a node's BF becomes 0 (its height did not change)
RR $(+2,+1)$	single Left rotation
LL $(-2,-1)$	single Right rotation
RL $(+2,-1)$	Right rotation on child, then Left at v
LR $(-2,+1)$	Left rotation on child, then Right at v
Rotations/insert	at most one (single or double)

Lecture 6: AVL Insert Correctness and Cost

1 What This Lecture Adds

Lecture 5 gave the **Insert** procedure by example and the **four rotation cases** (RR, LL, RL, LR). This lecture does *not* re-teach those. It proves the procedure restores the AVL property in **every** case, states the **node layout**, and shows each rotation is $\Theta(1)$.

Setup: the first unbalanced node

Insert x as a BST leaf, then walk up. Let A = the **first** node on the path from x to the root with $|\mathrm{BF}(A)| = 2$. Everything strictly below A is still AVL.
By symmetry assume $\mathrm{BF}(A) = +2$ (the -2 case is a mirror image). Then before the insert $\mathrm{BF}(A) = +1$, and the insert went into A's **right** subtree and **increased its height** from h to $h+1$. A's left subtree still has height h.

Let B = right child of A (root of the subtree that grew to height $h+1$).

2 Why the Insert Algorithm is Correct

2.1 The Skipped Step: $\mathrm{BF}(B) \neq 0$

Before rotating we must know B's shape. Since A is the **first** unbalanced node, B is still balanced: $\mathrm{BF}(B) \in \{-1, 0, +1\}$.

Claim: $\mathrm{BF}(B)$ cannot be 0

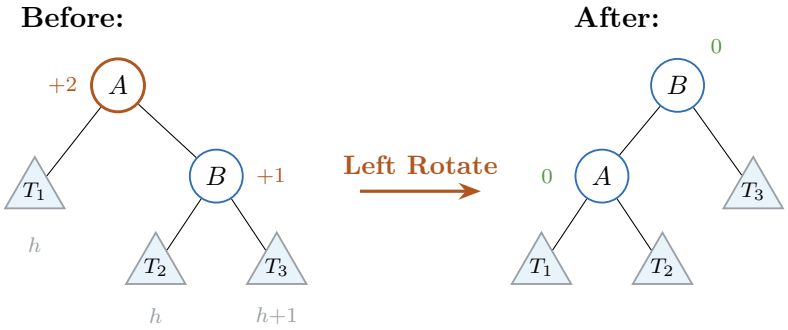
So $\mathrm{BF}(B) = +1$ (**Case 1a**) or $\mathrm{BF}(B) = -1$ (**Case 1b**). These are the only two cases.

Why $\mathrm{BF}(B) = 0$ is impossible. The insert of a single leaf increased B's height by exactly 1 (from h to $h+1$). One leaf can lengthen only **one** of B's two subtrees. If $\mathrm{BF}(B) = 0$ now (both subtrees height h), then before the insert one subtree was height h already, so B had height $h+1$ *before* the insert too. That contradicts "B's height increased." □

$\mathrm{BF}(B) = +1$: the growth is on B's **right** (right-right, RR).
$\mathrm{BF}(B) = -1$: the growth is on B's **left** (right-left, RL). Here name $C = B$.left.

2.2 Case 1(a): Single Left Rotation

x landed in B's right subtree. Fix with a single **Left Rotation at** A.



What the rotation does

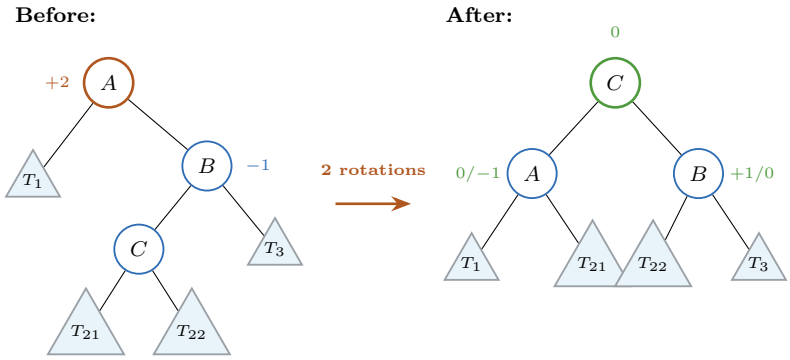
B rises, A becomes B's left child, and B's old left subtree T_2 (height h) becomes A's new right subtree. Order $T_1\,A\,T_2\,B\,T_3$ is unchanged, so the **BST property holds**. New factors: $\mathrm{BF}(A) = 0$, $\mathrm{BF}(B) = 0$.

Bonus: height is restored

Before inserting x, the subtree at A had height $h + 2$. After the rotation it is $h + 2$ again. **Same height as before the insert** $\Rightarrow$ no ancestor's BF changes $\Rightarrow$ stop. One rotation finishes the whole insert.

2.3 Case 1(b): Double Right-Left Rotation

x landed in B's **left** subtree, rooted at $C = B$.left. A single left rotation would **not** fix it. Do two: **right-rotate at** B, then **left-rotate at** A.



What the double rotation does

C (the middle key) rises to the top. Its two subtrees T_{21}, T_{22} split: T_{21} goes under A, T_{22} goes under B. Order $T_1\,A\,T_{21}\,C\,T_{22}\,B\,T_3$ is preserved, so the **BST property holds**. Final factors: $\mathrm{BF}(C) = 0$; A and B get $\{0, -1\}$ and $\{+1, 0\}$ depending on which of T_{21}, T_{22} received x. The subtree height returns to $h + 2$, so again we stop.

Special case $h = -1$

When $h = -1$, the subtrees are empty and C **is** the inserted leaf x. The double rotation collapses to: A, x, B become root x with children A and B, all BF $= 0$. Same rule, no special handling needed.

3 Cost, Node Layout, and Algorithm

3.1 Rotation Cost and Total Insert Cost

A rotation is $\Theta(1)$

A left (or right) rotation rewires only **3 pairs of pointers**. No subtree is copied or traversed $\Rightarrow \Theta(1)$.

Insert is $\Theta(\log n)$

BST insert of the leaf	$O(\log n)$
Walk up adjusting BF	$O(\log n)$, $O(1)$ per node
Rebalance	**at most one** rotation (single or double), $\Theta(1)$

The "height restored" bonus is what caps it at **one** rotation: after rotating, the subtree has its pre-insert height, so nothing above changes. Total: $O(\log n)$.

3.2 Contents of an AVL Node

AVL node layout

Each node stores **5 fields**:

PARENT	pointer to parent (needed to walk up)
`key`	the element's key (data)
`BF`	balance factor $\in \{-1, 0, +1\}$ (stored, not recomputed)
LEFT CHILD	pointer to left child
RIGHT CHILD	pointer to right child

Storing `BF` in the node lets the walk-up update it in $O(1)$ per step instead of recomputing subtree heights.

3.3 Final Insert Algorithm

Insert(T, x)

```
Insert x into T as in any BST; x is now a leaf; set BF(x) = 0.
Go up from x to the root.  For each node v on the path:
   Adjust BF(v):
      if x is in RIGHT subtree of v:  BF(v) += 1
      if x is in LEFT subtree of v:  BF(v) −= 1
      if BF(v) == 0:  STOP                                  (height unchanged)
   Rebalance:
      if BF(v) == +2:
          if BF(v.right)==+1:  Left Rotation; update BFs; STOP
          if BF(v.right)==−1:  Right-Left Rotation; update BFs; STOP
      if BF(v) == −2:  symmetric (Right or Left-Right rotation)
```

4 Exam-Style Practice

★ Practice: insert triggering a double rotation

Start from the AVL tree of $\{10, 20, 30, 40, 50\}$ built in order, then **Insert 25**. Show the rebalancing.

Inserting $10, 20, 30$ forces one left rotation, giving root 20. After $40, 50$ (another left rotation at 30) the tree is:

$$20 \to \{\, 10,\ 40 \to \{30, 50\}\,\}$$

Insert 25: search path $20 \to 40 \to 30$, land as **left child of 30**.
Walk up: $\mathrm{BF}(30) : 0 \to -1$, $\mathrm{BF}(40) : 0 \to -1$, $\mathrm{BF}(20) : +1 \to +2$.
$v = 20$ has $\mathrm{BF} = +2$ and $\mathrm{BF}(v.\mathrm{right}) = \mathrm{BF}(40) = -1 \Rightarrow$ **Right-Left rotation**: right-rotate at 40, then left-rotate at 20. Middle key 30 rises:

$$30 \to \{\, 20 \to \{10, 25\},\ 40 \to \{\emptyset, 50\}\,\}$$

All $|\mathrm{BF}| \le 1$. In-order: $10, 20, 25, 30, 40, 50$ ✓

AVL questions in this course ask you to **draw the tree after an insert/delete and name each rotation** (see the Fall 2014 boxes in Lecture 5). The skill tested is exactly the case decision above: $\mathrm{BF}(v)$ plus the heavy child's BF picks single vs double.

Quick Reference: Lecture 6

First unbalanced node A	lowest node with $\lvert\mathrm{BF}\rvert = 2$ after insert; assume $+2$
$B = A$.right	grew to height $h + 1$; $\mathrm{BF}(B) \neq 0$ (proved)
Case 1a $(+2, +1$, RR)	single Left rotation; $A, B \to 0$
Case 1b $(+2, -1$, RL)	Right-Left double; C rises, $\mathrm{BF}(C) = 0$
$h = -1$ special case	inserted leaf becomes the new subtree root
Rotation cost	3 pointer pairs $\Rightarrow \Theta(1)$
Height after fix	equals pre-insert height $\Rightarrow$ at most 1 rotation
Node fields	PARENT, key, BF, LEFT CHILD, RIGHT CHILD
Insert total	BST insert + walk-up + ≤ 1 rotation $\to \Theta(\log n)$

End of Lectures 1–6 notes.

Lecture 7: Augmenting Data Structures

1 Augmenting a Data Structure

Augmenting a data structure

Take an **existing** data structure and add **extra stored information** so it supports **new operations**, without breaking the old ones. Three steps:

1. Decide **what extra info** to store at each node.
2. Check the info can be **cheaply maintained** under every existing operation.
3. Use the info to **implement the new operations** efficiently.

Not automatic: choosing the right field takes **creativity**.

2 Dynamic Order Statistics

Dynamic Order Statistics ADT

A dynamic set S of elements with **distinct keys**, supporting:

- **Search**(x), **Insert**(x), **Delete**(x) (the Dictionary operations)
- **Select**(k): return the k-th smallest element (the element of **rank** k)
- **Rank**(x): given a pointer to x, return its rank

Rank of x = position of x in the sorted order of S (starting at 1).

2.1 Example

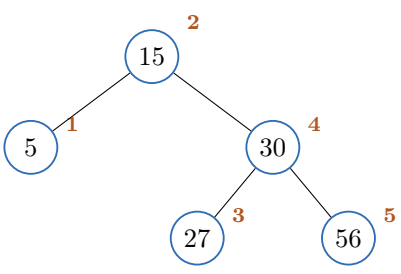
Select and Rank on $S = \{5, 15, 27, 30, 56\}$

$$\textbf{Select}(4) = 30 \quad \textbf{Select}(1) = 5 \quad \textbf{Rank}(15) = 2 \quad \textbf{Rank}(30) = 4$$

Rank(30) = 4 because **exactly 3** elements are < 30 (namely $5, 15, 27$), so 30 sits in position 4.

2.2 Naive Attempt: Store Each Node's Rank

Keep S in an **AVL tree** (Search/Insert/Delete already $O(\log n)$). To get Select/Rank, augment it. First idea: at each node store its **rank**.



Good, but...

Good: Rank(x) reads the stored field ($O(1)$ after finding x), and Select(k) searches for the node whose rank is k in $O(\log n)$.

Insert(2) (a new minimum) makes 2 rank 1 and **shifts every other rank up by 1**. One insert can rewrite **all** n rank fields $\Rightarrow$ Insert and Delete become $O(n)$. The rank field is **too global** to maintain cheaply.

3 Size-Augmented AVL Tree

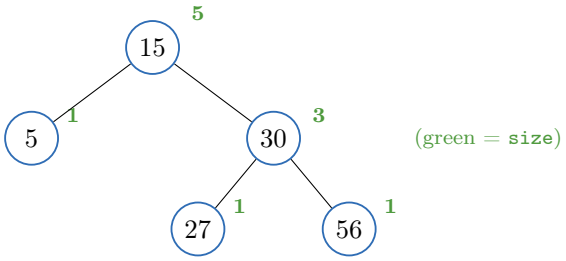
3.1 The `size` Field

Store subtree size

At each node x store $\texttt{size}(x)$ = number of nodes in the subtree rooted at x. Invariant, for every node:

$$\texttt{size}(x) = \texttt{size}(\text{left}(x)) + \texttt{size}(\text{right}(x)) + 1$$

(An empty child contributes $\texttt{size} = 0$.) This is a **local** field: it depends only on x's two children, so a single insert changes it on just **one root-to-leaf path**.



3.2 Select: the k-th Smallest

Relative rank RR(x)

RR(x) = the rank of x counting **only the nodes in x's own subtree**. In other words, if you sorted just x's subtree, RR(x) is the position x would land in.

$$\text{RR}(x) = \texttt{size}(\text{left}(x)) + 1$$

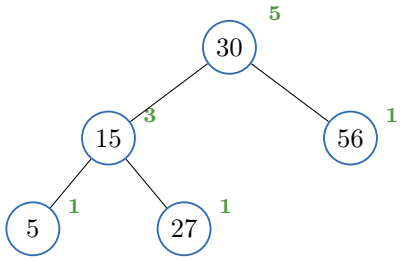
Why that formula works

In a BST, the **left subtree of x holds exactly the elements smaller than x** (within x's subtree). So:

- $\texttt{size}(\text{left}(x))$ = how many elements are smaller than x here.
- $+1$ = counts x itself.

Add them and you get x's position. No traversal needed: the `size` field already has the count.

Reading RR straight off the tree



Green numbers are `size`. Read RR off each node:

- RR(30) = `size`(left=15) + 1 = 3 + 1 = 4. So 30 is the **4th smallest** in this whole subtree. Check: 5, 15, 27, **30**, 56 ✓
- RR(15) = `size`(left=5) + 1 = 1 + 1 = 2. So 15 is the **2nd smallest** in the subtree under 15 ({5, 15, 27}). Check: 5, **15**, 27 ✓
- RR(5) = 0 + 1 = 1 (no left child). 5 is smallest in its subtree. ✓

Notice RR is **local**: RR(15) = 2 is its rank inside {5, 15, 27}, not its rank 2 in the full set. Select stitches these local ranks together as it walks down.

Select(x, k): the k-th smallest in the subtree at x

```
RR ← size(left(x)) + 1
if k = RR:  return x                                  // x sits exactly at position k
if k < RR:  return Select(left(x), k)                 // target is smaller, look left
if k > RR:  return Select(right(x), k − RR)           // skip x and all of left
```

Call `Select(T, k) = Select(root, k)`.

The one tricky line: why subtract when going right

Going right means the answer is bigger than x, so it is **not** x and not anyone in x's left subtree. That is RR elements we are leaving behind. Inside the right subtree those elements no longer exist, so the thing that was rank k overall becomes rank $k - \text{RR}$ *there*. That is why we recurse with $k - \text{RR}$, not k.

Trace: Select(3) on $S = \{5, 15, 27, 30, 56\}$

1. At root 15: RR = `size`(left = 5) + 1 = 1 + 1 = 2. $k = 3 > 2 \Rightarrow$ go right, $k \leftarrow 3 - 2 = 1$.
2. At 30: RR = `size`(left = 27) + 1 = 1 + 1 = 2. $k = 1 < 2 \Rightarrow$ go left.
3. At 27: RR = 0 + 1 = 1. $k = 1 = \text{RR} \Rightarrow$ return 27.

Select(3) = 27 ✓ (sorted: 5, 15, **27**, 30, 56).

Cost of Select

Each call drops one level (or returns), $O(1)$ per level, height $O(\log n) \Rightarrow O(\log n)$.

3.3 Rank of a Node

Given a pointer to x, walk **from x up to the root**, accumulating the elements that come before x.

Rank(T, x): rank of x in the whole tree

```
RR ← size(left(x)) + 1                          // rank of x within its own subtree
y ← x
while y ≠ root:
    if y is the RIGHT child of its parent p:
        RR ← RR + size(left(p)) + 1              // p and its left subtree precede x
    y ← p
return RR
```

Why

Climbing to parent p: if we came up from p's **right** subtree, then p itself and all of p's **left** subtree are smaller than x, so add $\texttt{size}(\text{left}(p)) + 1$. If we came up from the **left**, those nodes are all larger, so RR is **unchanged**.

Accumulating up the path

Suppose $\text{RR}(x) = 7$ inside x's subtree. Climbing through ancestors where x sits in the right subtree, with left-subtree sizes 3, then 6:

$$7 \;\to\; 7 + 3 + 1 = 11 \;\to\; 11 + 6 + 1 = 18.$$

Each right-turn adds "that ancestor + its left subtree." Left-turns add nothing.

Cost of Rank

One step per level, $O(1)$ each, height $O(\log n) \Rightarrow O(\log n)$.

3.4 Maintaining `size` on Insert and Delete

Insert(T, x): keep `size` correct

```
Phase 1 (BST insert):  insert x as a leaf; size(x) = 1;
    for each node y on the path from the root down to x:  size(y) += 1
Phase 2 (rebalance):  walk up adjusting BF; rotate if some BF hits ±2.
    After a rotation, recompute size() of the rotated nodes
    bottom-up:  size(z) = size(left(z)) + size(right(z)) + 1
```

Why it stays $O(\log n)$

Phase 1 touches one root-to-leaf path ($O(\log n)$ increments). A rotation only changes the children of **2 nodes**, so recomputing their `size` from the invariant is $O(1)$ extra. AVL insert already does at most one rotation $\Rightarrow$ total $O(\log n)$. **Delete** is symmetric (decrement down the path; fix `size` after each rotation).

Fixing `size` after a left rotation

A left rotation at A (right child B rises). Only A and B changed children, so recompute **bottom-up**:

- A (now lower): $\texttt{size}(A) = \texttt{size}(T_1) + \texttt{size}(T_2) + 1$.
- B (now on top): $\texttt{size}(B) = \texttt{size}(A) + \texttt{size}(T_3) + 1$.

Everything below T_1, T_2, T_3 is untouched, so two updates suffice.

4 Summary and Practice

Size-augmented AVL: all operations $O(\log n)$

Search / Insert / Delete	$O(\log n)$	AVL + fix `size` on the path
Select(k)	$O(\log n)$	descend using RR = `size`(left) + 1
Rank(x)	$O(\log n)$	climb, add `size`(left(p)) + 1 on right-turns

★ **Practice: Select and Rank**

Augmented AVL for $S = \{5, 15, 27, 30, 56\}$ (root 15; 30 has children 27, 56). Do **Select**(5) and **Rank**(27).

Select(5)**:** root 15, RR = 2, $5 > 2 \Rightarrow$ right, $k = 3$. At 30, RR = 2, $3 > 2 \Rightarrow$ right, $k = 1$. At 56, RR = $1 = k \Rightarrow$ return 56.
Rank(27)**:** start RR = `size`(left(27)) + 1 = 0 + 1 = 1. Climb to 30: 27 is 30's **left** child $\Rightarrow$ unchanged. Climb to 15: 30 is 15's **right** child $\Rightarrow$ RR $\leftarrow 1 +$ `size`(left(15)=5) + 1 = 1 + 1 + 1 = 3. Rank(27) = 3 ✓

Quick Reference: Lecture 7

Augmenting (3 steps)	pick field; keep it cheap to maintain; use it for new ops
Order statistics	Dictionary + Select(k) (k-th smallest) + Rank(x)
Naive (store rank)	Select/Rank easy but Insert/Delete $O(n)$ (too global)
Fix: store `size`(x)	`size`(x) = `size`(left) + `size`(right) + 1 (local)
RR(x)	`size`(left(x)) + 1 = rank of x within its subtree
Select(x, k)	=RR: return; <: go left; >: go right with $k -$ RR
Rank(T, x)	climb to root; right-child step adds `size`(left(p)) + 1
Maintain `size`	+1 down insert path; recompute 2 nodes per rotation ($O(1)$)
All operations	$O(\log n)$